门外谈 v2 改动清单 ——「信息层」改造（精确到函数）

> 侦察对象：`/Users/shf/Documents/DeepSeek harness/sixiangjia web`
>
> 关键文件：`app/page.tsx`（1516 行，前端全部逻辑）、`app/api/chat/route.ts`（374 行，圆桌/对谈）
>
> 辅助：`app/revolution/revolution.css`（样式）、`lib/skills.ts` / `lib/characterMeta.ts`（角色元数据）
>
> 本文档为**只读侦察产出**，不改动任何代码文件。
>
> 行号基于当前工作区版本（git HEAD = `517f2c0`）。

---

0. 总览

0.1 v2 要引入的三个机制

机制	一句话定义	落地位置
可见性 visibility	每条发言带 `public` / `private(to:某id)`；某发言者生成回复时，喂给它的历史只含它可见的发言	`route.ts:225-231` 的 `history` 构造改为按 viewer 过滤的工厂
悄悄话 whisper	用户单独对某位在座人物说话，其他人看不到；该人物后续每轮都带着这个前提发言	`page.tsx:975` 消息构造 + `page.tsx:981-985` 轮次循环 + `route.ts:315` 发言者选取
点名拉人对峙 summon	A 从悄悄话得知「B 不赞同你」→ A 公开回应时点名 B → 下一轮 B 提前发言	`route.ts:326-347` prompt 追加点名协议 + `route.ts:348-351` 解析 + `page.tsx:869-887` 下一轮顺序

0.2 改动点编号总表（共 26 个）

后端 `app/api/chat/route.ts`（A1—A11，11 个）

编号	位置	一句话
A1	`route.ts:200` 前（新建）+ `route.ts:34-38` 常量区	新增 `Vis` 类型、`isVisibleTo()`、`buildHistoryFor()`、`INFO_LAYER_`、*`SUMMON_`*、`SUMMON_RE`、`WHISPER_MARK`
A2	`route.ts:203`、`route.ts:206-214`	请求体解构加 `turnOrder?`、`whisperTo?`，并做白名单校验

编号	位置	一句话
A3	`route.ts:225-231`	`history` 常量 → `buildHistoryFor(viewerId)` 函数，按可见性过滤
A4	`route.ts:239-251`	`memeBlock` 改成 public-only 触发 + 按发言人计算，堵住现有泄漏
A5	`route.ts:270-283`	`streamOnce()` 增加「首行暂存」以拦截点名标签（方案 B，可选）
A6	`route.ts:285-305`	`ask()` 签名扩展，`who` 事件加 `whoId`，返回值带 `summon`
A7	`route.ts:309-314`	单人对谈分支改为走 `speakers[0]`（保证 whisper 到唯一在场者时不走错分支）
A8	`route.ts:315-353`	圆桌循环 `selected` → `speakers`；`debateContext` 用过滤后历史；`history.push` 带 `who` / `vis`
A9	`route.ts:326-347`	圆桌 system prompt 追加「信息差」与「点名」两条纪律
A10	`route.ts:348-351` 内	解析并剥离 `[[call:id]]`，emit `{type:"summon"}`
A11	`route.ts:203` + `route.ts:315`	悄悄话回合路由：`whisperTo` 命中时只让该角色发言，但**仍走圆桌分支**

前端 `app/page.tsx`（B1–B15，15 个）

编号	位置	一句话
B1	`page.tsx:9`	`type Msg` 加 `mid` / `whoId` / `vis` / `to`
B2	`page.tsx:404-409` 附近	新增 `whisperTo` state、`summonRef`、`summonTip` state
B3	`page.tsx:869-887`	`streamChat(current, whisperTo?)`；body 加 `turnOrder` / `whisperTo`
B4	`page.tsx:920-944`	SSE 分支加 `summon` / `whisper`
B5	`page.tsx:922`	`live` 记录 `whoId`（**当前 `Msg.who` 存的是显示名，不是 id**）
B6	`page.tsx:975-978`	`userMsg` 带 `vis` / `to` / `mid`
B7	`page.tsx:981-985`	轮次循环传 `whisperTo`、应用 `summonRef` 排序、循环后清理
B8	`page.tsx:796-816`、`817-823`	`saveChat` 标题回退 + `loadChat` 对旧数据容错
B9	`page.tsx:994-1003`	`/api/log` 脱敏，别把悄悄话传出去
B10	`page.tsx:1225-1234`	消息可见性标记（类名 + 标签 + 点名标签剥离）

编号	位置	一句话
B11	`page.tsx:1248-1255`（+ 可选 `1139-1156`、`1131`）	「私下说」目标条入口
B12	`page.tsx:99-379`	I18N zh/en 各补一批文案键
B13	`revolution.css:117-125` 附近	`.rv-msg-private`、`.rv-whisper-bar` 样式
B14	`page.tsx:976`、`844-859`	`maybeEscalate` / `userCount` 口径确认（悄悄话算不算一轮）
B15	`page.tsx:174-188`、`314-328`	changelog 补 v2 条目（可选）

0.3 三条不可动的回归红线

1. **`selected.length === 1` ⇒ 对谈模式**：`route.ts:309` 的分支条件是「在座一人」，**不是** `mode`。任何新逻辑都不能让 whisper 回合落进这个分支。
2. **SSE 事件是增量兼容的**：`page.tsx:920-944` 是 `if / else if` 链且**没有 `else`**，未知 `ev.type` 被静默忽略。所以加 `summon` / `whisper` 新事件对旧客户端零影响；反过来旧服务端对新客户端也安全。
3. **`/api/chat` 只有一个调用方**（`page.tsx:872`），`streamChat` 只有一个调用点（`page.tsx:983`）。契约可以放心扩，但要保持新增字段全部可选。

---

1. v2 数据契约（先定契约，再改代码）

1.1 前端 `Msg` 扩展（`page.tsx:9`）

```
// 现状
type Msg = { role: string; who?: string; color?: string; content: string; auto?: boolean;
err?: boolean };

// v2
type Vis = { kind: "public" } | { kind: "private"; to: string };   // to = 角色 id，或字面量
"user"
type Msg = {
  role: string;
  who?: string;         // ⚠ 现状存的是「显示名」（鲁迅 / Lu Xun），保留用于渲染，不要改语义
  whoId?: string;       // v2 新增：稳定角色 id，可见性判断只认它
  color?: string;
  content: string;
  auto?: boolean;
  err?: boolean;
  mid?: string;         // v2 新增：稳定消息 id（用 crypto.randomUUID），供后续 speaker→speaker 私聊
引用
  vis?: Vis;            // v2 新增：缺省 = public（旧存档天然兼容）
  to?: string;          // v2 新增：仅用户悄悄话消息，= 目标角色 id（便于 UI 直接读，等价 vis.to）
};
```

1.2 请求体扩展（`POST /api/chat`）

```
{
  characters: string[],        // 现状：入座名单。v2 语义 = 本轮「可用发言者集合」(保持全量，别用来表达 whisper)
  messages: Msg[],             // 现状：全量历史。v2 语义 = 全量（含 private)，由服务端过滤后再喂模型
  mode, scene, apiKey, baseURL, model, userId, lang,   // 全部不变
  // ---- v2 新增（全部可选） ----
  turnOrder?: string[],        // 本轮发言顺序；缺省 = characters 原序
  whisperTo?: string,          // 本轮为悄悄话回合：只让该角色发言（仍按圆桌规则）
  allowSummon?: boolean,       // 是否允许点名；缺省 = false，由前端显式传 true
}
```

> **为什么 `characters` 不能拿来表达 whisper**：`route.ts:309` 用 `selected.length === 1` 判断对谈模式。若悄悄话时只传一个 id，就会掉进单人分支（没有 `roundRules`、没有点名纪律、temperature 也从 0.85 变 0.8)。必须走 `whisperTo` 旁路。

1.3 SSE 事件扩展（服务端 → 前端）

现有事件（`route.ts`）：`who`(L286) / `delta`(L280) / `reset`(L292,296) / `note`(L249) / `done`(L303) / `error`(L356) / `end`(L354)。

v2 新增：

```
{ type: "summon", from: "<角色id>", fromName: "<显示名>", target: "<角色id>", targetName: "<显示名>" }
{ type: "whisper", to: "<角色id>", name: "<显示名>" }        // 悄悄话回合开场，前端可显示提示条
```

`who` 事件扩展一个字段（不删不改）：`{ type:"who", who:<id>, whoId:<id>, name:<显示名>, color }`。

---

2. 后端改动点逐条（`app/api/chat/route.ts`）

A1 —— 新增可见性内核与 v2 常量

位置

- 常量区：`route.ts:34-38`（`roundRules` 之后，`// ROUND_FN_END` 前后）
- 工具函数区：`route.ts:200`（`export async function POST` 之前）

现状：文件里没有任何「可见性」概念；`history` 是纯线性数组，所有发言人对它一视同仁（`route.ts:225-231`）。

改成什么

```
// ---- v2 信息层 ----
type Vis = { kind: "public" } | { kind: "private"; to: string };
type RawMsg = { role: string; who?: string; whoId?: string; content?: string; vis?: Vis |
string };

/** 这条发言对 viewerId 是否可见。缺省/异常一律视为 public（旧存档兼容）。 */
function isVisibleTo(m: RawMsg, viewerId: string): boolean {
  const v = m.vis;
  if (!v || v === "public") return true;
  if (typeof v === "string") return v === "public" || v === viewerId;   // 防御：有人直接传 id
字符串
  if (v.kind === "public") return true;
  if (v.kind === "private") return v.to === viewerId || v.to === "user";
  return true;
}

/** 按 viewer 过滤 + 保留现有 debate 前缀逻辑（原 route.ts:225-231）。 */
function buildHistoryFor(viewerId: string, messages: RawMsg[], mode: string, intl: boolean)
{
  return messages
    .filter((m) => isVisibleTo(m, viewerId))
    .map((m) => ({
      role: m.role === "assistant" ? "assistant" : "user",
      content:
        mode === "debate" && m.role === "assistant" && m.who
          ? (intl ? m.who + ": " : m.who + "说：") + String(m.content ?? "")
          : String(m.content ?? ""),
    }));
}

const INFO_LAYER_ZH = "\n\n【信息差】你只能看到上面这些发言。别人私下说过什么、你不在场时发生的事，你都不
知道——绝不假装知道。如果你的判断依赖于你不掌握的信息，就直接说你不知道，或者当场追问。";
const INFO_LAYER_EN = "\n\n[Information asymmetry] You only have the statements above.
Whatever was said privately to others, or while you were out of the room, you do not know —
never pretend to. If your judgement depends on information you lack, say so plainly, or ask
on the spot.";
const SUMMON_ZH = (roster: string) => "\n\n【可点名】在座者及其 id：" + roster + "。如果你认为其中
某人必须当场对质，请在你回答的**最后单独一行**输出 [[call:<id>]]（只输出这一行，别加解释）。只在你真要当面对
峙时才输出；对方会在下一轮最先发言。不点名就完全不要出现这个标记。";
const SUMMON_EN = (roster: string) => "\n\n[Call-out] At the table: " + roster + ". If
someone must be confronted right now, put [[call:<id>]] alone on the FINAL line of your
reply. Use it only for a real confrontation; they will speak first next round. Otherwise
never output that token.";
const SUMMON_RE = /\[\[\s*call\s*:\s*([A-Za-z0-9_-]+)\s*\]\]/;

/** 返回剥离后的文本 + 合法目标 id；目标非法时视为未点名。 */
function extractSummon(text: string, valid: Set<string>, selfId: string): { text: string;
target: string | null } {
  const m = SUMMON_RE.exec(text);
  if (!m) return { text, target: null };
  const id = m[1];
  const clean = text.replace(SUMMON_RE, "").replace(/\n{3,}/g, "\n\n").trimEnd();
  if (!valid.has(id) || id === selfId) return { text: clean, target: null };
  return { text: clean, target: id };
}
```

```
const WHISPER_MARK_ZH = "【私下只告诉你的话，别人听不到】\n";
const WHISPER_MARK_EN = "[Said to you in private — nobody else can hear this]\n";
```

牵连影响

- 纯新增，不触碰对谈 / 双语 / SSE。`buildHistoryFor` 只有在 A3 接线后才生效。
- `SUMMON_*` 只在圆桌分支使用（A9），对谈模式不加。
- 注意：`SUMMON_RE` 用了 `g` 会带 `lastIndex` 状态，**不要在函数外用 `.test()`**；上面用 `exec` + 字符串 `replace`，安全。

回归测试点

- 未传 `vis` 的老消息（`messages` 里全是旧格式）→ `buildHistoryFor` 输出必须与改造前 `route.ts:225-231` **逐字符一致**（可临时打日志对比）。

风险与坑

- `vis` 是前端可控字段，服务端必须把它当作**不可信输入**做形状校验（至少 `to` 必须在 `selected` 白名单内），否则用户能伪造成「只有某某能看见」的任意发言。

A2 —— 请求体解构与校验

位置：`route.ts:203`（解构）、`route.ts:206-214`（校验）

现状

```
const { characters: charIds, messages, mode, apiKey = "", baseURL = "", model = "", scene =
"casual", userId = "", lang = "zh" } = await req.json();
const intl = lang === "en";

if (charIds === undefined || Array.isArray(charIds) === false || charIds.length === 0) {
  return Response.json({ error: "请选择至少一位人物" }, { status: 400 });
}
const selected = charIds.map((id: string) => getMeta(id)).flatMap((m) => (m === undefined ?
[] : [m]));
if (selected.length === 0) {
  return Response.json({ error: "找不到对应人物" }, { status: 400 });
}
```

改成什么

```
const { characters: charIds, messages, mode, apiKey = "", baseURL = "", model = "",
        scene = "casual", userId = "", lang = "zh",
        turnOrder: rawOrder, whisperTo: rawWhisper } = await req.json();   // ← 新增两个
const intl = lang === "en";
// ... 现有校验不动 ...
const idSet = new Set(selected.map((s) => s.id));
const turnOrder: string[] = Array.isArray(rawOrder)
  ? rawOrder.filter((id: unknown): id is string => typeof id === "string" && idSet.has(id))
```

```
  : selected.map((s) => s.id);
const whisperTo: string | null = typeof rawWhisper === "string" && idSet.has(rawWhisper) ?
rawWhisper : null;
```

改成什么（发言者集合） —— 紧跟其后：

```
// 本轮实际发言的人：悄悄话回合只留目标；否则按 turnOrder 重排
const ordered = turnOrder
  .map((id) => selected.find((s) => s.id === id))
  .filter((s): s is NonNullable<typeof s> => !!s);
const speakers = whisperTo
  ? ordered.filter((s) => s.id === whisperTo)
  : (ordered.length ? ordered : selected);
const solo = selected.length === 1;          // ⚠️ 对谈模式仍然只看「在座一人」
```

牵连影响

- `turnOrder` 缺省时逐字等于 `selected`，行为不变。
- `solo` 与 `speakers.length` 解耦，是 A7/A11 的前提。

回归测试点

- 不传 `turnOrder` / `whisperTo` 时，`speakers` 必须 `=== selected`（同序、同长度）。
- 传了不属于 `selected` 的 id → 被静默过滤，不 500。
- 传 `turnOrder: []` → 回退到 `selected`（别让圆桌变成零人无输出）。

风险与坑

- `selected` 里可能有 `undefined` 已被 `flatMap` 过滤掉，`turnOrder` 的映射要再过滤一次（上面已做），否则 `speakers` 里出现 `undefined`，`speaker.id` 直接 TypeError → 触发 `route.ts:355` 的 catch 只吐 `error` 事件，前端一片空白。

A3 —— `history` 改成按 viewer 过滤（v2 的心脏）

位置：`route.ts:225-231`

现状

```
const history = messages.map((m: any) => ({
  role: m.role === "assistant" ? "assistant" : "user",
  content:
    mode === "debate" && m.role === "assistant" && m.who
      ? (intl ? m.who + ": " : m.who + "说：") + String(m.content ?? "")
      : String(m.content ?? ""),
}));
```

这是一个**模块级的共享可变数组**，`route.ts:350` 每轮 `history.push(...)` 往里塞，所有发言人读同一份。

改成什么

```
// 保留一条「原始消息」列表（可变，供轮内追加），每次给某个发言人组装 prompt 时再过滤
const rawHistory: RawMsg[] = (Array.isArray(messages) ? messages : []).map((m: any) => ({
  role: m.role === "assistant" ? "assistant" : "user",
  who: typeof m.who === "string" ? m.who : undefined,
  whoId: typeof m.whoId === "string" ? m.whoId : undefined,
  content: String(m.content ?? ""),
  vis: m.vis,                       // 原样带过来，isVisibleTo 负责兜底
}));
```

调用点（原 `route.ts:312` / `318`）改为：

```
...buildHistoryFor(selected[0].id, rawHistory, mode, intl)        // 单人分支
const debateContext = buildHistoryFor(speaker.id, rawHistory, mode, intl);   // 圆桌
```

⚠️ 原 `route.ts:318-325` 的 `debateContext` 里那段「从 `X说：` 反解角色、再按 intl 换名」的逻辑，要**搬进 `buildHistoryFor` 的 map 之前或之内**（因为它依赖 `who` 展示名）。建议把 `route.ts:318-325` 那段整体保留在圆桌循环里，只是把它的输入从 `history` 换成 `buildHistoryFor(...)`：

```
const debateContext = buildHistoryFor(speaker.id, rawHistory, mode, intl).map((m) => {
  const mm = /^([^\s：:]+)说[：:]\s*/.exec(m.content);
  if (m.role === "assistant" && mm) {
    const meta = getMeta(mm[1]);
    return { ...m, content: (intl ? (meta?.name || mm[1]) + ": " : (meta?.zhName || mm[1]) +
"说：") + m.content.slice(mm[0].length) };
  }
  return m;
});
```

改成什么（悄悄话对自己的标注） —— 在 `buildHistoryFor` 里补一条，让被私聊的人知道「这是私下的」：

```
// 插在 map 的 content 计算之前
if (m.vis && typeof m.vis === "object" && m.vis.kind === "private" && m.vis.to === viewerId)
{
  return { role: "user" as const, content: (intl ? WHISPER_MARK_EN : WHISPER_MARK_ZH) +
String(m.content ?? "") };
}
```

牵连影响

- **对谈模式（`selected.length === 1`）**：只剩一个 viewer，`isVisibleTo` 对 public 全放行；只有当用户曾对别人悄悄话（此时在座者已变）才会有差异。要保证「在座一人 + 历史里存在发给已离席者的悄悄话」不会让模型突然看到 `【私下…】` 标记而困惑——建议 viewer 不在 `selected` 的 private 消息一律按 public 放行？**不**：更安全的做法是**保持过滤**（离席者本就不该在场），但要加回归用例覆盖。

- **双语（intl / lang）**：`buildHistoryFor` 接收 `intl`，前缀逻辑与原实现逐字一致；`WHISPER_MARK_*` 分中英两版，别混。
- **SSE**：本改动不产生新事件。
- **`send()` 事件类型**：无关。

回归测试点

1. 全部 public 时，`buildHistoryFor(id, rawHistory, "debate", false)` 输出 == 改造前 `history`。
2. 仅「甲可见」的发言：`buildHistoryFor("甲", …)` 含它，`buildHistoryFor("乙", …)` 不含它。
3. `vis` 为 `undefined` / `"public"` / `{kind:"public"}` 三种形状都放行。
4. `mode === "chat"` 时 assistant 消息不加 `X说：` 前缀（保持 `route.ts:228` 的 `mode === "debate"` 条件）。
5. `intl === true` 时前缀是 `X:` 且 `who` 用的是 `meta.name`（英文清洗名）。

风险与坑

- `history` 原本既是「输入」又是「输出缓冲」（`route.ts:350` push）。改完后要**明确区分** `rawHistory`（含本轮新增）与「组装给模型的 messages」。漏改任何一处 `history` 引用 → 悄悄话泄漏给全场。
- 每轮每个人都重新过滤一遍全量历史，**token 成本随秘密数量线性上涨**；长会话（12 条存档上限 × 多轮）要在 A4 之后实测。
- 前端传上来的 `vis.to` 可能是**显示名**（因为 `Msg.who` 是显示名，很容易顺手写错）。契约必须写死：**`vis.to` 只接受角色 id**；服务端 A2 已做白名单，非法即视为 public，静默降级。

A4 —— `memeBlock` 的信息泄漏（现存 bug，v2 必须修）

位置：`route.ts:239-251`，注入点在 `route.ts:311`、`route.ts:330`

现状

```
let memeBlock = "";
const lastMsg = [...messages].reverse().find((m: any) => m.role !== "assistant");
const lastText = lastMsg ? String(lastMsg.content ?? "") : "";
if ((intl ? intlShould(lastText) : shouldSearch(lastText)) && allowSearch(userId)) {
  const info = intl ? await intlMemeSearch(...) : await fetchMemeInfo(...);
  if (info) {
    memeBlock = intl ? "\n\n[Field intel ...]" : "\n\n【场外情报...】";
    try { send({ type: "note", text: info.brief }); } catch {}
  }
}
```

`memeBlock` 只算一次，却拼进**每一位**发言人的 system（`route.ts:311`、`route.ts:330`）。

改成什么

```
// 只在「公开消息」上触发；悄悄话不触发联网，避免私密内容变成全员情报
const lastMsg = [...messages].reverse().find((m: any) => m.role !== "assistant" &&
isVisibleTo(m, "__public__") === true && (!m.vis || m.vis === "public" || m.vis.kind ===
"public"));
```

> 若产品上确实想让悄悄话也触发检索（被私聊的人自己去查），则改成**按发言人惰性计算**：
>
> ```ts
> const memeCache = new Map<string, string>();
> async function memoBlockFor(viewerId: string, viewerLastText: string) { ... }
> ```
>
> 代价：每位发言人一次搜索，且有 `allowSearch(userId)` 每 60 秒 2 次的节流（`route.ts:57-66`），多人圆桌会直接吃满配额。

推荐：v2 采取「**只在公开消息上触发**」这一最小改动。悄悄话内容是私密前提，不该外溢成公开情报。

牵连影响

- `{type:"note"}` 事件（`route.ts:249`）→ 前端 `page.tsx:933` `setMemeNote` → 渲染在消息列表顶部的唯一横幅（`page.tsx:1222-1224`）。这个横幅**用户看得到，是好事**，但它是「全局」的，不要试图让它跟随某个发言人。
- 对谈模式同样受影响，改动是纯收敛（少搜一次），可接受。
- 双语：`intlShould` / `intlMemeSearch` 分支不动。

回归测试点

1. 公开提问「XX 是什么梗」→ 依然触发检索，`note` 横幅照常出现。
2. 把同一句话用悄悄话发出 → **不触发**检索（网络面板无请求、无 note 事件）。
3. 悄悄话里含 `[field intel]` 字样不得出现在任何其他发言人的回复里。

风险与坑

- `route.ts:241` 的 `[...messages].reverse().find(...)` 会把**任何**最后一条非 assistant 消息当作「当前话题」。v2 之后前端会在悄悄话回合把私聊消息放在末尾 → 若不加过滤，私密文本会通过 `memeBlock` 泄给全场。这是**最容易漏的一个洞**。

A5 —— `streamOnce()` 首行暂存（方案 B，可选）

位置：`route.ts:270-283`

现状

```
const streamOnce = async (prompt: any[], guard: boolean): Promise<{ text: string; cjk:
boolean }> => {
  const res = await call(prompt);
  let text = "";
  let cjk = false;
  for await (const chunk of res) {
    const delta = chunk.choices?.[0]?.delta?.content || "";
    if (!delta) continue;
    text += delta;
    if (CJK_RE.test(delta)) cjk = true;
    if (guard && cjk) return { text, cjk: true };
    send({ type: "delta", text: delta });        // ← 逐字外发，无法回收
  }
  return { text, cjk };
};
```

改什么：把 `[[call:id]]` 约定放在**回答最开头**时，`streamOnce` 可以先攒住直到见到第一个 `\n`，判定首行是不是标签，再决定外发内容：

```
const streamOnce = async (prompt: any[], guard: boolean, holdFirstLine = false) => {
  // ... 同上 ...
  let held = "";
  let released = !holdFirstLine;
  const flush = (s: string) => { if (s) send({ type: "delta", text: s }); };
  for await (const chunk of res) {
    const delta = chunk.choices?.[0]?.delta?.content || "";
    if (!delta) continue;
    text += delta;
    if (guard && CJK_RE.test(delta)) return { text, cjk: true };
    if (!released) {
      held += delta;
      const nl = held.indexOf("\n");
      if (nl >= 0) {
        const first = held.slice(0, nl);
        const rest = held.slice(nl + 1);
        released = true; held = "";
        if (!SUMMON_RE.test(first)) flush(first + "\n");
        flush(rest);
      } else if (held.length > 64) {          // 兜底：超长首行，直接放行
        released = true; flush(held); held = "";
      }
    } else flush(delta);
  }
  if (!released && held) flush(held);
  return { text, cjk };
};
```

关联：`SUMMON_RE` 带 `g` 又用了 `.test()`——**必须**改成不带 `g` 的独立正则 `const SUMMON_TEST = /\[\[\scall\s:/;`，否则 `lastIndex` 会跨调用漂移导致偶发漏拦。

牵连影响

- 若采用方案 B，`prompt` 里点名纪律必须改成「放在**最开头**单独一行」。A9 的常量文案要同步改（本文档 A9 给的是「末尾」版本，二选一）。
- 英文防中文重试路径（`route.ts:293`、`297`）调用 `streamOnce` 时不要开 `holdFirstLine`，否则重试时 `reset` 与暂存冲突。
- 首字延迟增加（等第一个 `\n`）→ 体感上「啊哈」式抢答会慢一拍。

推荐：**先不做 A5**，采用「客户端渲染期剥离 + 服务端 `history` 剥离」的双保险（B4+B10），把 A5 当体验优化留到 v2.1。理由：改动面小、不会碰英文重试路径、验收简单。

回归测试点（若做）

1. 不点名时，`delta` 分片序列与改造前**逐片一致**（可以在客户端打日志比对）。
2. 点名时，前端从头到尾收不到 `[[call:` 字样。
3. 英文模式 C→中 拦截（`reset` / `EN_RETRY` / `EN_TRANSLATE`）三跳仍正常。

风险与坑

- 首行暂存会与 `guard` 提前 return（`route.ts:279`）交叉：提前 return 时 `held` 里可能有未外发内容。必须在 return 前 `flush(held)`，否则英文模式下会丢字。
- 别把 `holdFirstLine` 加到单人分支（`route.ts:310`）——对谈模式没有点名。

A6 —— `ask()` 扩展：`whoId` + 点名解析 + 悄悄话开场事件

位置：`route.ts:285-305`

现状

```
const ask = async (speaker: { id: string; zhName: string; name: string; color: string },
prompt: any[]) => {
  send({ type: "who", who: speaker.id, name: intl ? speaker.name : speaker.zhName, color:
speaker.color });
  if (intl) pinEnglish(prompt);
  let out = await streamOnce(prompt, intl);
  if (intl && out.cjk) { /* reset + EN_RETRY + EN_TRANSLATE */ }
  send({ type: "done", who: speaker.id });
  return out.text;
};
```

改成什么

```
const ask = async (
  speaker: { id: string; zhName: string; name: string; color: string },
  prompt: any[],
  opts: { vis?: Vis; allowSummon?: boolean; valid?: Set<string>; holdFirstLine?: boolean } =
{},
): Promise<{ text: string; summon: string | null }> => {
  send({ type: "who", who: speaker.id, whoId: speaker.id, name: intl ? speaker.name :
speaker.zhName, color: speaker.color });
```

```
  if (intl) pinEnglish(prompt);
  let out = await streamOnce(prompt, intl, opts.holdFirstLine === true);
  if (intl && out.cjk) { /* 原样保留 route.ts:291-302，内部调用不加 holdFirstLine */ }
  let text = out.text;
  let summon: string | null = null;
  if (opts.allowSummon) {
    const r = extractSummon(text, opts.valid || new Set(), speaker.id);
    text = r.text; summon = r.target;
  }
  send({ type: "done", who: speaker.id });
  return { text, summon };
};
```

牵连影响

- `ask` 的调用点有 1 处（`route.ts:310` 单人、`route.ts:348` 圆桌），返回值从 `string` 变对象 → 两处都要改。单人分支写 `(await ask(...)).text`。
- `who` 事件多一个 `whoId` 字段：老前端读 `ev.name`（`page.tsx:922`），不受影响。
- **`{type:"done"}` 里塞的是 `who: speaker.id`（角色 id），而前端当前完全没用它**（`page.tsx:934-935` 只 `pushLive`）。B5 会把 `whoId` 从 `who` 事件取，`done` 的 `who` 保持不动以免破坏兼容。
- 双语：`extractSummon` 中英通用（标签是 ASCII）。
- SSE：新增 `summon` 由**调用方**而不是 `ask` 发（保证提示条排在 A 的回复之后），见 A8。

回归测试点

1. `allowSummon` 未传 / false 时，返回的 `text` 与改造前**逐字符一致**（标签不会被误删）。
2. `intl` 重试三跳后 `summon` 仍能正确解析（重试返回的英文文本里若带标签，也要剥）。
3. `who` 事件里 `who` 与 `whoId` 同时存在且相等。

风险与坑

- 若模型把标签写成 `[[call: 鲁迅]]`（中文名）或 `[[CALL:x]]` → 正则不匹配。`SUMMON_RE` 保持严格（只认 id）更好：让「没解析到」表现为「这轮不点名」，而不是错误点名。
- **点名标签会进入 `turns`/持久化路径**：`route.ts:349` `turns.push({ who: speaker.id, text })` 和 `route.ts:350` `history.push(...)` 必须用**剥离后**的 `text`，否则标签会被下一轮模型看到并被模仿（自我强化）。

A7 —— 单人分支改为 `speakers[0]`

位置：`route.ts:309-314`

现状

```
if (selected.length === 1) {
  const singleText = await ask(selected[0], [
    { role: "system", content: getSkillSystem(selected[0].id, intl) + memeBlock + (intl ?
ANTI_OUT_EN : ANTI_OUT) + (...) },
```

```
    ...history,
  ]);
  replies.push({ who: selected[0].id, text: singleText });
}
```

改成什么（行为必须不变）

```
if (solo) {                                    // 见 A2：solo === (selected.length === 1)
  const only = speakers[0] || selected[0];
  const r = await ask(only, [
    { role: "system", content: getSkillSystem(only.id, intl) + memeBlock + (intl ?
ANTI_OUT_EN : ANTI_OUT) + (intl ? (SCENE_RULES_EN[scene] || SCENE_RULES_EN.casual) :
(SCENE_RULES[scene] || SCENE_RULES.casual)) + (intl ? EN_ONLY : LANG_FOLLOW) },
    ...buildHistoryFor(only.id, rawHistory, mode, intl),
  ]);
  replies.push({ who: only.id, text: r.text });
}
```

牵连影响

- 对谈模式**不加** `roundRules`、不加 `INFO_LAYER`、不加 `SUMMON`、temperature 保持 `0.8`（`route.ts:258` 的条件是 `selected.length === 1`，不用改）。
- 悄悄话回合若在座只有 1 人（用户对唯一的人私聊），会走到这里。产品上「私下说」在 `selected.length < 2` 时应被 UI 禁用（B11），服务端保持这个兜底即可。

回归测试点

1. 单人对谈 + 中文 casual/serious/heated 三档：输出长度纪律不变。
2. 单人对谈 + 英文：`pinEnglish`、CJK 拦截、`reset`/重试仍生效。
3. 单人对谈 **不出现**「可点名」纪律文案（grep system 字符串）。

A8 —— 圆桌循环：`selected` → `speakers` + 过滤历史 + 带可见性入栈 + 发点名事件

位置：`route.ts:315-353`（核心循环 `route.ts:317`；`debateContext` `route.ts:318-325`；`turns.push` / `history.push` `route.ts:349-350`）

现状

```
} else {
  const turns: { who: string; text: string }[] = [];
  for (const speaker of selected) {                        // ← 顺序 = 入座顺序，写死
    const debateContext = history.map(...);                // ← 全员共享历史
    const prompt = [ { role:"system", ... }, ...debateContext, { role:"user", ... } ];
    const text = await ask(speaker, prompt);
    turns.push({ who: speaker.id, text });
    history.push({ role: "assistant", content: text });   // ← 不带 who，轮内发言无署名
  }
```

```
  replies.push(...turns);
}
```

改成什么

```
} else {
  const turns: { who: string; text: string }[] = [];
  if (whisperTo) {
    const target = speakers.find((s) => s.id === whisperTo);
    if (target) send({ type: "whisper", to: target.id, name: intl ? target.name :
target.zhName });
  }
  for (const speaker of speakers) {
    const debateContext = buildHistoryFor(speaker.id, rawHistory, mode, intl).map((m) => {
      /* 原 route.ts:318-325 的反解/换名逻辑，逐字保留 */
    });
    const roster = speakers.map((s) => (intl ? s.name : s.zhName) + "=" + s.id).join(", ");
    const summonRule = opts.allowSummon ? (intl ? SUMMON_EN(roster) : SUMMON_ZH(roster)) :
"";
    const prompt = [
      { role: "system", content:
          getSkillSystem(speaker.id, intl) + memeBlock +
          (intl ? roundRules(true, speaker) : roundRules(false, speaker)) +
          (intl ? INFO_LAYER_EN : INFO_LAYER_ZH) + summonRule +          // ← v2 新增
          (intl ? ANTI_OUT_EN : ANTI_OUT) +
          (intl ? (SCENE_RULES_EN[scene] || SCENE_RULES_EN.casual) : (SCENE_RULES[scene] ||
SCENE_RULES.casual)) +
          (intl ? EN_ONLY : LANG_FOLLOW) },
      ...debateContext,
      { role: "user", content: /* 原 route.ts:339-345 逐字保留 */ },
    ];
    const { text, summon } = await ask(speaker, prompt, {
      allowSummon: !!opts.allowSummon, valid: idSet, holdFirstLine: false,
    });
    turns.push({ who: speaker.id, text });
    rawHistory.push({
      role: "assistant",
      who: intl ? speaker.name : speaker.zhName,     // ← 顺带修「轮内发言无署名」
      whoId: speaker.id,
      content: text,
      vis: { kind: "public" },
    });
    if (summon) {
      const t = speakers.find((s) => s.id === summon);
      send({ type: "summon", from: speaker.id, fromName: intl ? speaker.name :
speaker.zhName,
             target: summon, targetName: t ? (intl ? t.name : t.zhName) : summon });
    }
  }
  replies.push(...turns);
}
```

牵连影响

- **顺序**：`speakers` 由 `turnOrder` 决定；缺省 = `selected` 原序（UI 拖动排序的结果，`page.tsx:720-742` `onDropCard`），行为不变。
- **`who` 署名**：`rawHistory.push` 加了 `who`，会让**同一轮内**后续发言人看到 `X说:` 前缀。现状是轮内无前缀（只有跨轮从 `messages` 恢复的才有）。这**改变了现有观感**（模型会更清楚谁说了什么），但也会影响双语反解（`getMeta(mm[1])` 需要 `mm[1]` 是中文名/英文名；两端都要能被 `getMeta` 命中）。**风险中等，建议单独一次改动 + 单独回归。**
- 保守方案：`who` 只在 `intl` 时装 `speaker.name`、中文时装 `speaker.zhName`，与 `route.ts:229` 的写法一致。
- 若不想引入任何行为变化，可暂时不写 `who`（保持现状），可见性仍靠 `vis`。**v2 最稳的选法。**
- **`whisper` 事件**：只在悄悄话回合发；前端 B4 用它插一条 `auto` 提示。
- **`summon` 事件**：由调用方发（在 `done` 之后），前端保证提示条排在 A 的回复之后。
- SSE 事件格式：`send()` 是 `route.ts:236-238` 的 `data: <JSON>\n\n`，前端按 `\n\n` 切分（`page.tsx:913-919`），新增事件沿用同一格式，无需改解析器。

回归测试点

1. 四位圆桌、不点名、不私聊 → 发言顺序与改造前**完全一致**；每条回复内容不应因为 `INFO_LAYER` 而改变语气（可能略有变化，属预期）。
2. `turnOrder` 反转 → 发言顺序反转。
3. 只对「甲」私聊 → 本轮**只有甲发言**；乙丙丁在后续轮次看不到私聊内容（给甲丙的 system 打日志核对）。
4. 点名 `乙` → 前端出现 `summon` 事件，且下一轮 `乙` 第一。
5. `allowSummon: false` → 模型输出里不会出现 `[[call:`，`summon` 事件不出现。
6. 圆桌 + `mode === "chat"`（前端允许 3 人 + chat 模式）：`buildHistoryFor` 不加前缀，与现状一致。
7. `send({type:"end"})` 仍在最后一轮之后发出（`route.ts:354`），前端 `page.tsx:941-944` 收尾。

风险与坑

- **顺序状态跨请求**：`send()` 的 `for (let r = 0; r < rn; r++)`（`page.tsx:982`）**每轮一次 HTTP**。点名后「下一轮 B 提前」必须由**前端**保存并回传（B7），服务端没有跨请求记忆。别在服务端写「上轮点过谁」的全局 Map——多用户会串台（现有代码里 `searchBuckets`（`route.ts:58`）就是模块级 Map，已经是个反面教材）。
- `rawHistory.push` 位置在 `ask` 之后：本轮后续人能看到本轮前人的话（现状如此，`route.ts:350`）。若改成「本轮前人的话对后人是 private」就不是 v2 范围了。
- `roster` 里带角色 id 会**显著加长 system prompt**（12 人时很可观）。建议只列 id + 中文名，或只在 `selected.length <= 6` 时启用点名。

A9 —— 圆桌 system prompt 追加两条纪律

位置：`route.ts:326-347`（插入到 `route.ts:331` 的 `roundRules` 之后、`ANTI_OUT` 之前）

现状：system 由 6 段拼成：`getSkillSystem` + `memeBlock` + `roundRules` + `ANTI_OUT` + `SCENE_RULES` + (`EN_ONLY` | `LANG_FOLLOW`)。

改成什么

```
... + (intl ? roundRules(true, speaker) : roundRules(false, speaker))
    + (intl ? INFO_LAYER_EN : INFO_LAYER_ZH)          // ← 信息差纪律
    + (opts.allowSummon ? (intl ? SUMMON_EN(roster) : SUMMON_ZH(roster)) : "")   // ← 点名协议
    + (intl ? ANTI_OUT_EN : ANTI_OUT)
    + ...
```

顺序建议：`roundRules`（圆桌纪律）→ `INFO_LAYER`（你知道什么）→ `SUMMON`（你能做什么）→ `ANTI_OUT`（不要出戏）→ `SCENE_RULES`（说多长）→ `EN_ONLY/LANG_FOLLOW`（说什么语言）。**语言类规则仍必须放最后**，与现状一致（`route.ts:334`）。

牵连影响

- 只影响圆桌；对谈模式（A7）不加。
- 双语：两份常量，`intl` 二选一。
- 会改变现有回复的语气/长度（模型会开始说「我不知道」「那你呢」）。**这是 v2 的预期效果，但要做 A/B 观感验收**。
- 英文模式：`EN_ONLY` 要求「不出现任何汉字」，而 `INFO_LAYER_ZH` 只在中文用，不冲突；`roster` 在英文下要用 `speaker.name`（英文清洗名，来自 `lib/skills.ts` 的 `named()`）。

回归测试点

1. 中文圆桌：回复里不会出现「根据我掌握的信息」这种客服腔（`ANTI_OUT` 仍在生效）。
2. 英文圆桌：回复中 CJK 字符数 = 0（现有 `CJK_RE` 拦截 + `reset` 路径仍生效）。
3. serious 档仍守「最多 6 句、300 字」（`route.ts:8`）。
4. 只选 1 人时，system 里 grep 不到 `信息差` / `可点名`。

风险与坑

- system 变长 → 弱模型（默认 `glm-4-flash`，`route.ts:4`）更可能忽略 `roundRules`，出现「请提供具体问题」这类跳出角色的话术。`ANTI_OUT`（`route.ts:12`）要保持在 `INFO_LAYER` **之后**。
- 点名协议要求「最后单独一行」时，模型可能把标签和正文写在**同一行**。解析用一个宽松正则（`SUMMON_RE` 不限行首）即可，但要防止正文里出现 `[[call:` 字样被误删——概率极低，接受。

A10 —— 解析并剥离点名标签（已并入 A6/A8）

位置：`route.ts:348-351` 之后

要点回顾

- 剥离发生在 `ask()` 返回前（A6），**未剥离的原文绝不进入 `turns` / `rawHistory`**。
- `send({type:"summon", ...})` 在 `rawHistory.push` **之后**发，保证事件顺序：`delta…` → `done` → `summon` → 下一人 `who…`。
- `target` 必须是 `idSet` 里的 id 且 `!== speaker.id`（`extractSummon` 已处理）。

风险与坑

- 前端 `pushLive()`（`page.tsx:898-901`）在 `who` 事件（`page.tsx:921`）里触发：**下一位发言人的 `who` 会把上一位的 `live` 固化**。所以 `summon` 事件若晚于下一位的 `who` 到达，`auto` 提示会插到错误位置。**必须在循环内、`await ask()` 返回后立刻发 `summon`，且在下一轮 `ask` 之前**（上面的 A8 代码已保证）。

A11 —— 悄悄话回合路由

位置：`route.ts:203`（读取 `whisperTo`）、`route.ts:315`（`speakers` 已收窄）

改成什么

- `whisperTo` 命中时 `speakers = [目标]`，但 `solo === false`（因为在座 ⩾ 2），所以**仍走圆桌分支**：有 `roundRules`、有 `INFO_LAYER`、有 `SUMMON`（这才是「A 被私聊后能点名 B」的前提）。
- 悄悄话回合的 `send({type:"whisper"})` 让前端插一条 `auto` 提示：「〔密〕 你私下对 鲁迅 说」。

牵连影响

- `selected.length === 1` 且 `whisperTo === 该人` → `solo === true`，走单人分支（无点名）。**可接受**，且 UI 层已禁用单人对谈的私下说入口（B11）。
- 前端 `rounds` 语义（1–4，`page.tsx:1182-1188`）：悄悄话回合算 1 轮。`page.tsx:981-985` 的 `rn` 不变，`r === 0` 时才算 whisper 回合（B7）。

回归测试点

1. 圆桌 + 悄悄话：本轮 SSE 只出现 1 组 `who…done`。
2. 悄悄话回合后紧接的普通回合，顺序回到 `selected`（除非有 `summon`）。
3. 悄悄话回合的回复是 **public**，下一轮其他人能看到（这是点名对峙的舞台）。

风险与坑

- 「reply 是 public」是产品决策，**必须写死并测试**：A 公开回应你的私房话 → 全场知道有人给 A 打过小报告 → 这正是戏剧张力来源。若改成「A 的回复也是 private」，点名机制就没有观众了，整个 v2 垮掉。
- 用户是**全知观众**：前端渲染全部消息（含私聊），靠 `vis` 标记区分。不要为了「保密」在前端隐藏私聊——用户自己说的话必须看得见。

---

3. 前端改动点逐条（`app/page.tsx`）

B1 —— `type Msg` 扩展

位置：`page.tsx:9`

```
// 现状
type Msg = { role: string; who?: string; color?: string; content: string; auto?: boolean;
err?: boolean };

// v2（与 §1.1 一致）
type Vis = { kind: "public" } | { kind: "private"; to: string };
type Msg = { role: string; who?: string; whoId?: string; color?: string; content: string;
             auto?: boolean; err?: boolean; mid?: string; vis?: Vis; to?: string };
```

牵连影响：`Msg` 被 `SavedChat`（`page.tsx:97`）、`messages`（`page.tsx:404`）、`streamChat`、`send`、`saveChat`、`/api/log` 共用。新增字段全可选 → 旧 `localStorage` 数据（`mw_chat_history`，`page.tsx:475-479`、`812`）无需迁移。

回归测试点：`tsc --noEmit` 通过；读取旧存档（含 12 条老对话）能正常渲染。

风险与坑：`Mid` 用 `crypto.randomUUID()`（Node/浏览器都支持，`page.tsx:441` 已有同款兜底写法）。但**同一消息在多次 `setMessages` 里会重复调用 `randomUUID`**（`bump()` 里 `{...live}` 复制）——`mid` 只在**创建时**生成一次（`userMsg` 和 `live` 初始化），复制时继承，不要每帧生成。

B2 —— 新增 state

位置：`page.tsx:404-409`（`messages` / `input` / `loading` / `mode` / `memeNote` 一带）

```
// 推荐放在 page.tsx:408 之后，与聊天态同组
const [whisperTo, setWhisperTo] = useState<string | null>(null);   // null = 公开
const [summonTip, setSummonTip] = useState<string | null>(null);   // 可选，或复用 auto msg
const summonRef = useRef<string | null>(null);                     // 跨轮次传递点名目标
```

派生值（放在 `page.tsx:464` `const t = I18N[lang];` 附近）

```
const whisperTarget = whisperTo ? charById(whisperTo) : null;      // charById 定义在
page.tsx:632，注意调用顺序！
const canWhisper = mode === "debate" && selected.length >= 2;
useEffect(() => { if (whisperTo && !selected.includes(whisperTo)) setWhisperTo(null); },
[selected, whisperTo]);
```

牵连影响

- `charById` 在 `page.tsx:632` 才声明（`const charById = (id) => ...`）。`whisperTarget` 若在 `page.tsx:464` 使用会**踩 TDZ**；把它挪到渲染区（`page.tsx:1127` 之后）或改写为内联 `chars.find(...)`。

- `summonRef` 是 ref（跨 `streamChat` 调用存活，且不触发重渲染），必须放在组件顶层。

回归测试点

- 离座（`toggleChar`，`page.tsx:633-636`）后 `whisperTo` 自动清空，输入框回到「公开」态。
- 切到对谈模式（`selected.length === 1`）时 `whisperTo` 清空、目标条消失。

风险与坑

- 不要把 `whisperTo` 塞进 `SavedChat`（`page.tsx:97`）——它是瞬时 UI 状态，不是会话状态。

B3 —— `streamChat` 签名与请求体

位置：`page.tsx:869-887`

现状

```
const streamChat = async (current: Msg[]): Promise<Msg[]> => {
  const ctrl = new AbortController();
  abortRef.current = ctrl;
  const res = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    signal: ctrl.signal,
    body: JSON.stringify({
      characters: selected,
      messages: current.map((m) => ({ role: m.role, content: m.content, who: m.who })),
      mode, scene,
      apiKey: localStorage.getItem("mw_api_key") || "",
      baseURL: localStorage.getItem("mw_base_url") || "",
      model: localStorage.getItem("mw_model") || "",
      userId: uid, lang,
    }),
  });
```

改成什么

```
const streamChat = async (current: Msg[], whisper?: string | null): Promise<Msg[]> => {
  const ctrl = new AbortController();
  abortRef.current = ctrl;
  // 点名后的下一轮顺序：被点名者提前
  const summoned = summonRef.current;
  const turnOrder = summoned
    ? [summoned, ...selected.filter((id) => id !== summoned)]
    : selected;
  summonRef.current = null;                    // ← 用完即清（顺序在 fetch 之前定好）
  const res = await fetch("/api/chat", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    signal: ctrl.signal,
    body: JSON.stringify({
      characters: selected,
```

```
      turnOrder,
      whisperTo: whisper || null,
      messages: current.map((m) => ({ role: m.role, content: m.content, who: m.who, whoId:
m.whoId, vis: m.vis })),
      mode, scene,
      apiKey: localStorage.getItem("mw_api_key") || "",
      baseURL: localStorage.getItem("mw_base_url") || "",
      model: localStorage.getItem("mw_model") || "",
      userId: uid, lang,
    }),
  });
```

牵连影响

- `whisperTo` 只在 `mode === "debate" && selected.length > 1` 时非空（由 B7 保证）。
- `m.who` 仍原样上传（服务端 `route.ts:228` 依赖它做前缀），新增 `whoId` 与 `vis`。
- 请求体变大（全量历史 + vis），但历史本来就全量上传（现状），增量很小。

回归测试点

1. 不点名的普通回合：`turnOrder === selected`，请求体多出的字段被服务端忽略（旧服务端也能跑，只是忽略）。
2. 点名回合：`turnOrder[0] === 被点名者`。
3. `summonRef` 在 `fetch` 前被清空 → 不会连续两轮都提前同一人。

风险与坑

- `summonRef.current = null` 的**位置**是坑：必须在构造 `body` **之前**取值、**之后**清空。若清在取值前 → 点名失效；若清在 `await fetch` 之后 → 若同一轮内又有新点名会互相覆盖（实际上不会，因为每轮只点一次，但语义要清楚）。
- `selected` 是 state（`page.tsx:397`）。`send()` 在异步循环里多次调用 `streamChat`，期间 `selected` 理论上可能被用户改动（拖动排序），会造成顺序抖动。现状就有这个问题，v2 不加剧，但**建议在 `send()` 开头快照**：`const seated = [...selected];` 并传给 `streamChat`。

B4 —— SSE 事件分支新增 `summon` / `whisper`

位置：`page.tsx:920-944`

现状

```
if (ev.type === "who") { ... }
else if (ev.type === "delta" && live) { ... }
else if (ev.type === "reset" && live) { ... }
else if (ev.type === "note") { setMemeNote(String(ev.text || "")); }
else if (ev.type === "done") { ... }
else if (ev.type === "error") { ... }
else if (ev.type === "end") { pushLive(); setMessages([...out]); break; }
```

链尾**没有** `else`，未知类型静默丢弃 → 纯增量扩展。

改成什么（插在 `note` 与 `done` 之间）

```
} else if (ev.type === "summon" && ev.target) {
  summonRef.current = String(ev.target);
  pushLive();                                    // 先把当前发言固化
  const nm = String(ev.targetName || ev.target);
  out = [...out, { role: "assistant", auto: true,
    content: lang === "zh" ? "⚔ " + String(ev.fromName || "") + " 点名 " + nm + " 当场对质——下一轮 " + nm + " 先开口"
                           : "⚔ " + String(ev.fromName || "") + " calls out " + nm + " — " + nm + " speaks first next round" }];
  if (!pausedRef.current) setMessages([...out]);
} else if (ev.type === "whisper") {
  const nm = String(ev.name || ev.to || "");
  out = [...out, { role: "assistant", auto: true,
    content: lang === "zh" ? "〔密〕 你对 " + nm + " 私下说（其他人听不到）" : "〔密〕 Whispered to " + nm + " (nobody else hears)" }];
  if (!pausedRef.current) setMessages([...out]);
}
```

牵连影响

- `pushLive()`（`page.tsx:898-901`）会把 `live` 追加到 `out`；`summon` 事件在服务端是 `done` **之后**发的，而 `done` 分支（`page.tsx:934-935`）已经 `pushLive()` 过。此处再调一次 `pushLive()` 是幂等安全的（`live` 已为 null）。**保守起见核对一下**：`done` 只 `pushLive()` 但**没有**重新渲染？——它调了 `setMessages([...out])`。所以 `summon` 分支只需追加 `out` 再 `setMessages`，可以省掉 `pushLive()`。实现时以「不重复插入」为准。
- `auto` 消息复用 `.rv-msg-auto` 样式（`revolution.css:487`、`661`），无需新 CSS。
- 双语：中英两版文案。
- **暂停（paused）语义**：`pausedRef`（`page.tsx:445`）在 `stopChat`（`page.tsx:861-867`）里复位。`auto` 提示走 `if (!pausedRef.current)` 与现有 `who` 分支一致（`page.tsx:923`）。

回归测试点

1. 老服务端（不发 `summon` / `whisper`）→ 前端一切照旧。
2. 收到未知类型（例如未来加的 `mute`）→ 不抛异常、不中断流。
3. `summon` 事件夹在两个 `who` 之间 → `auto` 提示位置正确（在 A 的回复后、B 的回复前）。

风险与坑

- `ev.target` 是**角色 id**，`ev.targetName` 是展示名。前端渲染**必须用** **`targetName`**（`Msg.who` 是展示名这套约定要保持一致）。
- 不要在 `summon` 分支里改 `live`，否则会污染当前气泡。

B5 —— `live` 记录 `whoId`（关键：`Msg.who` 是显示名，不是 id）

位置：`page.tsx:920-923`

现状

```
if (ev.type === "who") {
  pushLive();
  live = { role: "assistant", who: ev.name, color: ev.color, content: "" };
  if (!pausedRef.current) bump();
}
```

服务端 `route.ts:286` 发的是 `{ who: speaker.id, name: <显示名>, color }` —— **`ev.who` 是 id，但前端只用了 `ev.name`**。所以 `Msg.who` = 「鲁迅」/「Lu Xun」，**不是** `luxun`。

改成什么

```
live = { role: "assistant", who: ev.name, whoId: String(ev.whoId || ev.who || ""), color:
ev.color, content: "", mid: crypto.randomUUID?.() ?? String(Date.now()) };
```

牵连影响

- `page.tsx:878` 上传 `m.whoId` → 服务端可用它做更稳的可见性归属（而非靠显示名反解）。
- `page.tsx:1230` 渲染 `{m.who}` 不变。
- `route.ts:318-325` 的反解逻辑仍读 content 里的 `X说:` 前缀，**不依赖 `whoId`**，所以现在就可以不改服务端反解。
- 历史存档：老消息无 `whoId`。**可见性判断不依赖 `whoId`**（只依赖 `vis`），所以老存档天然 public，安全。

回归测试点

1. 中英切换后 `who` 仍显示正确语言的名字。
2. 圆桌 4 人发言，每条 assistant 消息的 `whoId` 都能在 `chars` 里找到。

风险与坑

- `Msg.who` 是显示名的这个事实，是 v2 最容易踩的坑：**任何要指向具体角色的地方（`vis.to`、`whisperTo`、`summonRef`）都必须用 id**。建议在 `Msg` 的注释里写死这条约定。

B6 —— `send()` 构造带可见性的用户消息

位置：`page.tsx:975-979`

现状

```
const userMsg: Msg = { role: "user", content: v };
const userCount = messages.filter((m) => m.role === "user").length + 1;
```

```
maybeEscalate(v, userCount);
let current = [...messages, userMsg];
setMessages(current); setInput(""); setLoading(true); setTyping(true);
```

改成什么

```
const target = canWhisper ? whisperTo : null;
const userMsg: Msg = target
  ? { role: "user", content: v, to: target, vis: { kind: "private", to: target },
      mid: crypto.randomUUID?.() ?? String(Date.now()) }
  : { role: "user", content: v, mid: crypto.randomUUID?.() ?? String(Date.now()) };
const userCount = messages.filter((m) => m.role === "user").length + 1;
maybeEscalate(v, userCount);
let current = [...messages, userMsg];
setMessages(current); setInput(""); setLoading(true); setTyping(true);
```

牵连影响

- **对谈模式**：`canWhisper === false` → `userMsg` 与现状一致（只是多了个 `mid`）。
- **`maybeEscalate`**（`page.tsx:844-859`）：悄悄话会照常触发气氛升级（含 `ESCALATE_WORDS` 命中）。**口径决策**：建议悄悄话**也**计数（`userCount` 就是 `messages` 里的 user 数，天然包含），因为「私房话最能点燃火气」。但要在回归里明确写出这是有意为之。
- **`saveChat(current)`**（`page.tsx:992`）：`vis` 会被一起存进 `localStorage`（`page.tsx:812`）。旧版本代码读这份数据会忽略 `vis`（无害）。
- **标题**：`page.tsx:800-803` `firstUser.content.slice(0, 24)` ——若第一条 user 消息是悄悄话，历史列表里会露出私密文本。建议 B8 处理。
- **`/api/log`**（`page.tsx:994-1003`）会把悄悄话原文上报 → B9 处理。

回归测试点

1. 公开发言：`userMsg.vis === undefined`，服务端/前端行为与改造前一致。
2. 悄悄话：气泡带「私下」标记（B10），且**不是** `role:"assistant"`。
3. 发送后 `whisperTo` 是否复位 —— 产品决策，见 B11。

风险与坑

- `vis` 一定要是**对象**（`{kind,to}`），不要偷懒写成 `vis: target` 字符串。`isVisibleTo`（A1）有字符串兜底，但服务端 A2 的校验路径会把它当非法输入。**契约统一成对象**。

B7 —— 轮次循环：传 `whisperTo` + 应用点名顺序

位置：`page.tsx:981-985`

现状

```
const rn = mode === "debate" ? rounds : 1;
for (let r = 0; r < rn; r++) {
  current = await streamChat(current);
  if (r < rn - 1) await new Promise((res) => setTimeout(res, 400));
}
```

改成什么

```
const rn = mode === "debate" ? rounds : 1;
const firstWhisper = canWhisper ? whisperTo : null;
for (let r = 0; r < rn; r++) {
  current = await streamChat(current, r === 0 ? firstWhisper : null);
  if (r < rn - 1) await new Promise((res) => setTimeout(res, 400));
}
summonRef.current = null;          // 收尾兜底
```

语义说明（要写进文档/验收）

- `r === 0` 且 `firstWhisper` 非空 ⇒ 本轮是悄悄话回合（只有目标发言，`rounds` 少跑一轮完整圆桌）。
- 后续 `r` 回到完整圆桌；若第 0 轮点名了 B，第 1 轮 B 先开口（`summonRef` 由 B4 设置、B3 消费）。
- **只有第 0 轮能私聊**：用户一次点击 = 一个私聊回合 + (rounds−1) 个完整回合。若想「私聊后再打一整轮」，把 `rn` 临时 +1 或循环结束后补一轮，属产品决策。

牵连影响

- `rounds` UI（`page.tsx:1182-1188`）显示 1–4；`rounds===1` 时悄悄话回合**就是全部** → 用户只会看到目标的回复。这是合理的。
- `setTyping` / `loading` 的 finally（`page.tsx:988-1007`）不变。

回归测试点

1. `rounds===2` + 无私聊 → 两轮完整圆桌（与现状一致）。
2. `rounds===2` + 私聊 → 第 1 个请求 `whisperTo` 非空且只有 1 个 `who`；第 2 个请求 `whisperTo === null` 且发言人数 = 在座数。
3. 私聊回合里点名 B → 第 2 个请求 `turnOrder[0] === B`。
4. 停止按钮（`stopChat`，`page.tsx:861-867`）在悄悄话回合中途按下 → `abortRef.current?.abort()` 生效，已说内容保留（`page.tsx:947-954`）。

风险与坑

- `streamChat` 里的 `sendSeqRef` 机制（`page.tsx:964`、`989`）用于防止「旧请求的 finally 覆盖新状态」。多次 `await` 的循环里运行良好，**不要在循环中引入新的 `setState` 时序依赖**。
- `summonRef.current = null` 放在循环后是兜底；真正的消费在 `streamChat` 内（B3）。若 `rn === 0`（不可能，`rounds >= 1`）会残留——已兜底。

B8 —— `saveChat` / `loadChat` 兼容

位置：`page.tsx:796-816`（`saveChat`）、`page.tsx:817-823`（`loadChat`）、`page.tsx:487-502`（恢复当前会话）

现状

```
const firstUser = msgs.find((m) => m.role === "user");
const entry: SavedChat = { id, title: firstUser ? firstUser.content.slice(0, 24) : ...,
  messages: msgs, mode, characters: selected, createdAt: Date.now(), updatedAt: Date.now()
};
```

改成什么

```
// 标题：跳过私密消息，避免历史列表直接暴露悄悄话内容
const firstUser = msgs.find((m) => m.role === "user" && !(m.vis && m.vis.kind ===
"private"));
const entry: SavedChat = { ..., title: firstUser ? firstUser.content.slice(0, 24) : (lang
=== "zh" ? "对话" : "Chat"), ... };
```

`loadChat`（`page.tsx:817-823`）**不需要改**：`vis` 随 `messages` 一起存取；缺失即 public。

牵连影响

- 旧存档（无 `vis`）读取后所有消息按 public 处理 → 与改造前完全一致。
- `localStorage` 配额（`page.tsx:812` 已有 try/catch）：`vis` 让每条消息增大约 30 字节，`slice(0, 12)` 上限 12 份会话，压力可忽略。

回归测试点

1. 用旧版本的 `mw_chat_history`（可从 git 历史里构造）恢复 → 无报错、无 `vis` 相关异常。
2. 悄悄话作为第一条 user 消息的会话 → 历史列表标题不是悄悄话原文。
3. `loadChat` 后继续发言 → 悄悄话的可见性仍生效（服务端按 `vis` 过滤）。

风险与坑

- **服务端不知道存档**：每次请求都全量重传 `messages`（`page.tsx:878`）。所以「历史被谁看过」这件事在服务端是**无状态**的，好改也好测——但意味着**过滤必须在每次请求里重做**，不能缓存。

B9 —— `/api/log` 脱敏

位置：`page.tsx:994-1003`

现状

```
fetch("/api/log", { method: "POST", headers: {...},
  body: JSON.stringify({ userId: uid, mode, characters: selected.map(...), messages: current
```

```
}) });
```

改成什么

```
messages: current.map((m) =>
  m.vis && m.vis.kind === "private"
    ? { role: m.role, who: m.who, content: "[private]", auto: m.auto, err: m.err }
    : m),
```

牵连影响

- 与 `/api/log` 的实现（`app/api/log/route.ts`）解耦：字段结构不变，只是内容替换。
- 对谈模式无影响。

回归测试点

1. 发出悄悄话 → 服务端日志里搜不到悄悄话原文。
2. 公开对话日志照常记录（`app/api/log/route.ts` 落盘的字段不变）。

风险与坑

- 悄悄话是**用户主动输入的内容**，从隐私角度更不该流向站长日志。若产品要求保留审计，则至少要哈希/截断，并在 README 里写明。

B10 —— 消息列表可见性标记

位置：`page.tsx:1225-1234`

现状

```
{messages.map((m, i) => (
  m.auto ? <div key={i} className="rv-msg-auto">{m.content}</div>
         : <div key={i} className={"rv-msg " + (m.role === "user" ? "user" : "ai") + (m.err ?
" rv-err" : "")} style={{ animationDelay: (0.05 * i) + "s" }}>
            {m.who && <div className="rv-msg-who" style={{ color: m.color }}>■ {m.who}
</div>}
            <div>{m.content}</div>
          </div>
))}
```

改成什么

```
const CALL_STRIP = /\[\[\s*call\s*:\s*[A-Za-z0-9_-]+\s*\]\]/g;      // 模块级常量，放
page.tsx:53 附近
const showContent = (s: string) => s.replace(CALL_STRIP, "").replace(/\n{3,}/g,
"\n\n").trimEnd();

{messages.map((m, i) => {
  const priv = !!(m.vis && m.vis.kind === "private");
```

```
  const toName = priv ? (lang === "en" ? (charById(m.vis!.to)?.name || m.vis!.to) :
(charById(m.vis!.to)?.zhName || m.vis!.to)) : "";
  return m.auto ? <div key={m.mid || i} className="rv-msg-auto">{m.content}</div>
    : <div key={m.mid || i}
        className={"rv-msg " + (m.role === "user" ? "user" : "ai") + (m.err ? " rv-err" :
"") + (priv ? " rv-msg-private" : "")}
        style={{ animationDelay: (0.05 * i) + "s" }}>
        {m.who && <div className="rv-msg-who" style={{ color: m.color }}>■ {m.who}</div>}
        {priv && <div className="rv-msg-vis">{lang === "zh" ? "▸ 私下 · 只有 " + toName + " 能
看见" : "▸ Private · only " + toName + " sees this"}</div>}
        <div>{showContent(m.content)}</div>
      </div>;
})}
```

牵连影响

- `showContent` 是 A5（方案 B）的**兜底**：即使服务端没能拦住 `[[call:x]]`，前端也不会把标签显示给用户。同时对**流式过程中**的 `live` 气泡生效（`bump()` 里 `live.content` 直接进 `messages` 渲染，`page.tsx:903`）。
- `key` 从 `i` 换成 `m.mid || i`：老消息无 `mid`，退回 `i`，行为不变。
- `charById` 在 `page.tsx:632` 声明 → 渲染函数在 `page.tsx:1036` 之后，作用域 OK。
- 双语：标签中英两版。

回归测试点

1. 公开消息：class 与 DOM 与改造前一致（无多出的 `rv-msg-vis`）。
2. 悄悄话消息：出现 `rv-msg-private` 与「只有 X 能看见」，X 是中/英对应的名字。
3. 若模型漏出 `[[call:luxun]]` → 界面不显示该标签（同时 B4 的 `summon` 提示正常）。
4. 打字机式的逐字渲染（`bump()`）不受影响，无闪烁。

风险与坑

- `m.vis!` 非空断言要在 `priv` 判定之后用；TS 严格模式下更稳的写法是 `const vis = m.vis && m.vis.kind === "private" ? m.vis : null;`。
- `charById(m.vis.to)` 里的 `to` 可能是字面量 `"user"`（未来的 speaker→user 私聊）→ `charById("user")` 返回 `undefined`，用 `|| m.vis.to` 兜底显示。**别让它渲染成 `undefined`**。

B11 —— 「私下说」入口（详见 §4）

见第 4 节。改动点落在 `page.tsx:1248-1255`（推荐）与 `page.tsx:1139-1156`（可选）。

B12 —— I18N 文案

位置：`page.tsx:99-379`（`I18N.zh` / `I18N.en`）

新增键（zh / en 各一份）

```
whisperTag: "私下",                    // "Private"
whisperTo: "只对",                     // "Only to"
whisperAll: "全体",                    // "Everyone"
whisperPh: "只对 {name} 说，其他人听不到…",   // "Whisper to {name} — nobody else hears…"
whisperSend: "私下说",                 // "Whisper"
whisperVis: "私下 · 只有 {name} 能看见", // "Private · only {name} sees this"
whisperTip: "你对 {name} 私下说（其他人听不到）",
summonTip: "{from} 点名 {target} 当场对质——下一轮 {target} 先开口",
whisperHint: "信息差：被私下告知的人记得你说的话，其他人不知道——这将改变圆桌的走向",
```

牵连影响

- `I18N` 用的是 `as const` 风格对象推断（`page.tsx:464` `const t = I18N[lang];`）。两个语言包**必须同时补齐同名键**，否则 `t.whisperTag` 在另一语言下是 `undefined`（TS 会报错，因为联合类型推断）。
- 若采用 `{name}` 占位符，需要一个小函数 `fmt(s, name)`（`String.replace`），别用模板字符串拼在 I18N 里。

回归测试点

- 切 EN/中，目标条、消息标签、点名提示条全部换语言，无残留中文/英文串。
- `tsc --noEmit` 通过（两侧键一致）。

风险与坑

- `I18N.en.changelog`（`page.tsx:314-328`）与 `I18N.zh.changelog`（`page.tsx:174-188`）是数组，长度不必一致（`page.tsx:1109` 用 `slice(0, 3)`，`page.tsx:1399` 全量）。加条目时注意别插到前 3 条之外否则首页「近日更新」看不到。

B13 —— CSS

位置：`app/revolution/revolution.css:117-125`

现状

```
.rv-msg { max-width: 80%; padding: var(--s-2) var(--s-3); font-size: 0.92rem; line-height:
1.8; opacity: 0; transform: translateY(12px); animation: rvUp 0.4s var(--ease-out) forwards;
}
.rv-msg.user { align-self: flex-end; background: var(--ink); color: var(--paper); }
.rv-msg.ai { align-self: flex-start; background: var(--paper-2); border: 1px solid var(--
line); color: var(--ink); }
.rv-msg.rv-err { border-color: var(--red); color: var(--red); }
.rv-msg-who { font-family: var(--font-display); font-size: 0.7rem; font-weight: 900; margin-
bottom: var(--s-1); letter-spacing: 0.08em; }
.rv-chat-input { display: flex; gap: var(--s-2); padding: var(--s-3); border-top: 2px solid
var(--ink); }
```

新增

```
/* 可见性：私下消息用红色左竖线 + 半透明底，与公开消息区分 */
.rv-msg.rv-msg-private { border-left: 3px solid var(--red); box-shadow: inset 0 0 0 1px
color-mix(in srgb, var(--red) 30%, transparent); }
.rv-msg.user.rv-msg-private { background: color-mix(in srgb, var(--ink) 78%, var(--red)); }
.rv-msg-vis { font-family: var(--font-serif); font-size: 0.66rem; letter-spacing: 0.1em;
opacity: 0.85; margin-bottom: var(--s-1); }
.rv-msg.user .rv-msg-vis { color: var(--paper); }

/* 私下说目标条 */
.rv-whisper-bar { display: flex; align-items: center; gap: var(--s-2); flex-wrap: wrap;
padding: var(--s-2) var(--s-3) 0; border-top: 2px solid var(--ink); }
.rv-whisper-bar .lb { font-family: var(--font-serif); font-size: 0.68rem; letter-spacing:
0.14em; color: var(--ink-soft); }
.rv-whisper-chip { border: 1px solid var(--line-strong); background: var(--paper); color:
var(--ink-soft); font-family: var(--font-serif); font-size: 0.72rem; padding: 0.25em 0.6em;
cursor: pointer; }
.rv-whisper-chip.active { border-color: var(--red); color: var(--red); box-shadow: inset 0
-2px 0 var(--red); }
.rv-chat-input.in-whisper input { border-color: var(--red); }
```

连带调整

- 目标条与 `.rv-chat-input` 都带 `border-top: 2px solid var(--ink)`。若两者相邻会出现双线。要么把目标条放进 `.rv-chat-input` 容器内（推荐，见 §4），要么把 `.rv-chat-input` 的 `border-top` 在非私聊态才加。
- 移动端断点（`revolution.css:600-603`、`774-777`）里 `.rv-chat-input { flex-wrap: wrap; }`：目标条 chips 要能换行，用 `flex-wrap: wrap` + `overflow-x: auto` 二选一，别把输入框挤没。

回归测试点

- 桌面 / 平板 / 手机（`revolution.css:580-603`、`754-777` 三档断点）下目标条不溢出、输入框不被压。
- 暗色/浅色？——本项目是纸感单主题（`--paper` / `--ink` / `--red`），无暗色模式，不用考虑。

风险与坑

- CSS 里 `.rv-msg` 有 `opacity: 0` + `animation` 入场（`revolution.css:117`）。新增的 `.rv-msg-private` **不要**覆盖 `opacity` / `animation`，否则消息会永久不可见。
- `color-mix` 在项目里已被使用（`revolution.css:852`），可放心用。

B14 —— `maybeEscalate` / `userCount` 口径

位置：`page.tsx:844-859`（`maybeEscalate`）、`page.tsx:976`（`userCount`）

现状

```
const maybeEscalate = (text: string, round: number) => {
  const i = SCENE_ORDER.indexOf(scene);
```

```
  if (i >= 2) return;
  const hit = ESCALATE_WORDS.some((w) => text.includes(w));
  if (hit || round >= 3) { ...升级 + 插入 auto 消息... }
};
```

v2 口径决策（需要在文档里明确写死）

- 悄悄话**计入** `userCount`（`messages.filter(role === "user").length + 1` 天然包含）→ 私房话也会推进气氛升级。**推荐保留**（悄悄话往往就是点火）。
- `ESCALATE_WORDS`（`page.tsx:53`）对悄悄话文本同样生效。**推荐保留**。
- 但要注意：升级提示（`page.tsx:853-857` 的 `auto` 消息）是**公开显示**的。如果用户在悄悄话里说「你错了」，界面会插入「 气氛上来了」——这条提示所有消息流都看得见。**这不算泄漏**（它不含悄悄话原文），可以接受。

回归测试点

1. 悄悄话触发升级 → `auto` 提示出现且**不含**悄悄话原文。
2. `scene` 到 `heated` 后不再升级（`if (i >= 2) return`，`page.tsx:846`）。

风险与坑

- `maybeEscalate` 里读的是 state `scene`（闭包捕获），而 `setScene` 是异步的。连续快速发送可能重复升级——**现存问题，v2 不修**，但改动 `send()` 时别加剧（例如不要在 whisper 分支里额外调用 `maybeEscalate`）。

B15 —— changelog（可选）

位置：`page.tsx:174-188`（zh）、`page.tsx:314-328`（en）

改成什么：在数组**开头**插入一条 v2 描述（因为 `page.tsx:1109` 只显示前 3 条）。

```
"信息层上线：悄悄话 + 信息差 —— 你可以单独对某位在座者说话，其他人听不到；他从这条私房话出发，还可能当场把反对者点名拉上台（下一轮对方先开口）",
```

风险与坑：`I18N` 是普通对象字面量，`changelog` 数组类型是 `string[]`，插入即可；中英各一条，措辞别太长（首页只显示 3 行）。

---

4. 前端 UI 改动点专节

4.1 「私下说」入口放哪

候选方案对比

方案	位置	优点	缺点
A. 头像上加入口	`page.tsx:1139-1156` 的 `.rv-avatar`，在 `.rm`（`×`）旁加一个 `私` 按钮	空间直觉强（"对这个人说"）	compact（5–11 人）/ mini（12+ 人）模式下 `.nm` / `.sc` **被隐藏**（`revolution.css:950-963`），按钮会被挤爆；`.rm` 已有 `margin-left: auto`，再加一个要重排
B. **输入区上方目标条**（推荐）	`page.tsx:1248-1255` `.rv-chat-input` 之前	常驻可见、与「说话」动作在同一视线、断点友好、不用碰角色卡逻辑	需要一点新 CSS；与 `.rv-chat-input` 的 `border-top` 要合并处理
C. 输入框内嵌 `@` 提及	`page.tsx:1249` 的 `<input>`	无需新 UI	要写一个 @ 补全；当前是单行 input 无富文本能力；体验成本最高

推荐 B，并补一条**轻量的 A 作为引导**（点头像上的「私」= 等价于在目标条里选中该人），但**只在 `!mini && !compact` 时渲染**。

具体 JSX（插在 `page.tsx:1248` 之前，作为 `.rv-chat-input` 的兄弟或父容器）

```
{canWhisper && (
  <div className="rv-whisper-bar">
    <span className="lb">{t.whisperTo}</span>
    <button className={"rv-whisper-chip" + (!whisperTo ? " active" : "")}
            onClick={() => setWhisperTo(null)}>{t.whisperAll}</button>
    {selected.map((id) => {
      const c = charById(id);
      if (!c) return null;
      return (
        <button key={id}
          className={"rv-whisper-chip" + (whisperTo === id ? " active" : "")}
          style={whisperTo === id ? { borderColor: c.color, color: c.color } : undefined}
          onClick={() => setWhisperTo((p) => (p === id ? null : id))}>
          {lang === "en" ? c.name : c.zhName}
        </button>
      );
    })}
    {whisperTo && <span className="lb">· {t.whisperHint}</span>}
  </div>
)}
```

输入框与按钮的联动（改 `page.tsx:1249-1252`）

```
<input ref={inputRef} value={input} onChange={(e) => setInput(e.target.value)}
  onKeyDown={(e) => { if (e.key === "Enter" && !(e.nativeEvent as any).isComposing) send();
}}
  placeholder={whisperTo && canWhisper ? t.whisperPh.replace("{name}", lang === "en" ?
(whisperTarget?.name || "") : (whisperTarget?.zhName || "")) : t.ph} />
...
```

```
<button className="rv-btn" onClick={loading ? stopChat : send} disabled={!loading &&
!input.trim()}>
  {loading ? (lang === "zh" ? "■ 停止" : "■ Stop") : (whisperTo && canWhisper ? t.whisperSend
: t.send)}
</button>
```

发送后是否复位：推荐**复位为公开**（`setWhisperTo(null)`，在 `send()` 的 `finally` 里或 `setInput("")` 旁）。理由：避免用户以为在公开说话、实际发出去了私房话。若想连续私聊，再点一次 chips 成本很低。

约束（必须实现）

```
const canWhisper = mode === "debate" && selected.length >= 2;    // 对谈/单人：无入口
useEffect(() => { if (whisperTo && !selected.includes(whisperTo)) setWhisperTo(null); },
[selected, whisperTo]);
```

4.2 消息列表怎么标记可见性

- **用户悄悄话**（`m.role === "user" && m.vis?.kind === "private"`）：
- class：`rv-msg user rv-msg-private`
- 顶部标签：`▸ 私下 · 只有 鲁迅 能看见`
- 视觉：红色左竖线 + 深色气泡里掺红（B13）
- 位置仍在右侧（`align-self: flex-end`，`revolution.css:118`）——「我说的话」永远是右侧，可见性靠竖线与标签区分，不靠左右。
- **发言者的公开回复**：不动。
- **（可选）发言者的私下回复**：`m.role === "assistant" && m.vis?.kind === "private"` → `rv-msg ai rv-msg-private`，标签 `▸ 私下对你说`。**v2 不做**，但 `Msg.vis` 已经能表达，留接口即可。
- **点名提示条**：`auto: true` 消息（`page.tsx:1226-1227`），复用 `.rv-msg-auto`，无需新样式。
- **悄悄话开场提示**：同上，`auto: true`。

4.3 状态放在哪个 `useState`

状态	类型	位置	生命期	
`whisperTo`	`useState<string \`	null>(null)`	`page.tsx:408` 之后	组件级；`selected` 变化时清理；`send()` 后复位
`summonRef`	`useRef<string \`	null>(null)`	`page.tsx:445-448` 一带（与 `pausedRef` / `abortRef` 同组）	跨 `streamChat` 的一次轮次循环；`streamChat` 消费后清空

状态	类型	位置	生命期	
`summonTip`（可选）	`useState<string | null>(null)`		`page.tsx:409` 旁边	**不推荐**——直接用 `auto` 消息更省事，且能进存档
`whisperTarget`	派生值，不用 state	渲染区（`page.tsx:1127` 之后，`charById` 已声明）	—	

为什么不把 `whisperTo` 放进 `messages`：`send()` 在 `page.tsx:975` 已经把目标写进了 `userMsg.vis`，那是**持久化**的真相；`whisperTo` 只是**输入区的 UI 意图**。两者职责分离，别合并。

---

5. 端到端时序（三个机制的完整走位）

5.1 悄悄话 → 点名对峙（主场景）

```
用户：选中 [鲁迅, 哈耶克, 凯恩斯], mode=debate, rounds=2
① 点目标条「鲁迅」→ whisperTo="luxun"
② 输入「哈耶克私下说你那套国民性批判早就过时了」→ Enter
   B6: userMsg = { role:"user", content:"哈耶克私下说…", to:"luxun",
                   vis:{kind:"private",to:"luxun"}, mid:"…" }
③ B7: r=0 → streamChat(current, "luxun")
   B3: turnOrder=[luxun,hayek,keynes]（无 summon）、whisperTo="luxun"
   A2: speakers=[luxun], solo=false（在座 3 人）
   A8: send({type:"whisper", to:"luxun", name:"鲁迅"})
       鲁迅的 system = SKILL + roundRules + INFO_LAYER + SUMMON(roster) + …
       鲁迅的 history = buildHistoryFor("luxun", …)
         → 含这条 private（且被 WHISPER_MARK_ZH 标注「私下只告诉你的话」）
       鲁迅发言：「哈耶克？让他自己站上来说。[[call:hayek]]」
   A6: extractSummon → text="哈耶克？让他自己站上来说。", summon="hayek"
   A8: rawHistory.push({role:"assistant", content:<剥离后>, vis:public})
       send({type:"summon", from:"luxun", fromName:"鲁迅", target:"hayek", targetName:"哈耶
克"})
   A8 循环结束（speakers 只有 1 人）→ send({type:"end"})
   B4: summonRef.current = "hayek"；插入 auto 提示「⚔ 鲁迅 点名 哈耶克…」
④ B7: r=1 → streamChat(current, null)
   B3: turnOrder=["hayek","luxun","keynes"], whisperTo=null
   A2: speakers=[hayek,luxun,keynes]（重排后）
   A8: 哈耶克第一个发言 —— 他自己**看不到**那条 private
       （buildHistoryFor("hayek", …) 过滤掉 vis.to==="luxun" 的消息）
```

关键验证：哈耶克的 system prompt 里不能出现「哈耶克私下说你那套…」这句原文。

5.2 对谈模式（`selected.length === 1`）

- 目标条不渲染（`canWhisper === false`）。
- 走 A7 单人分支：无 `roundRules`、无 `INFO_LAYER`、无 `SUMMON`、temperature 0.8。
- SSE 只有 `who…delta…done`，然后 `end`。
- **验收：改造前后输出风格无差异。**

5.3 双语

- `whisper` / `summon` 事件的展示名由服务端按 `intl` 给（`route.ts:286` 同一套规则）。
- `WHISPER_MARK_*` 服务端按 `intl` 二选一。
- 前端标签走 `I18N[lang]`。
- **EN_ONLY（`route.ts:18`）优先级最高**：英文模式下即使用户用中文写悄悄话，被私聊者的回复仍必须是纯英文。悄悄话原文是 user 消息，不受 `EN_ONLY` 约束（`EN_ONLY` 只约束模型输出）。

---

6. 回归测试清单（手工可操作）

6.1 对谈模式（`selected.length === 1`）—— 最高优先级

- □ 1 人 + 随口：「你好」→ 回复 ≤ 2 句 / ≤ 60 字（`route.ts:7`）
- □ 1 人 + 较真：回复 ≤ 6 句 / ≤ 300 字（`route.ts:8`）
- □ 1 人 + 上头：正常展开
- □ 1 人 + EN：零 CJK 字符；故意用中文提问，验证 `reset`（`page.tsx:928-931`）清屏后英文版本补上
- □ 1 人 + EN：查梗触发英文检索（`route.ts:243` `intlShould`），`note` 横幅出现
- □ 1 人：目标条**不渲染**；输入框 placeholder 仍是 `t.ph`
- □ 1 人：SSE 只有 `who` / `delta` / `done` / `end`（可在 Network → EventStream 里看）

6.2 圆桌（`selected.length >= 2`）

- □ 3 人 + rounds=2 + 不私聊：两轮，每轮 3 人，顺序 = 侧栏入座顺序
- □ 拖动侧栏头像改顺序（`page.tsx:720-742`）→ 发言顺序跟随
- □ rounds=4：四轮完整圆桌
- □ 圆桌 + EN：全员英文，零 CJK，`debateContext` 的 `X:` 前缀正确（`route.ts:322`）
- □ 圆桌 + `mode === "chat"`（3 人但选「对谈」按钮）：历史不加 `X说:` 前缀（`route.ts:228` 条件不满足），其余不变
- □ 12 人 mini 模式（`page.tsx:548`）：`roster` 拼出的 system 不导致超长/超时（90s timeout，`route.ts:220`）

6.3 可见性

- □ 对甲私聊后，**乙丙的回复里不出现私聊原文**（打日志比对 3 份 `buildHistoryFor` 输出）
- □ 私聊后进入下一轮，甲**仍然**记得私聊内容（信息不对称是持久的，不是一次性的）
- □ `vis` 缺省 / `"public"` / `{kind:"public"}` 三种形状都被放行
- □ 伪造：前端手动把 `vis.to` 改成一个不在座的人 → 服务端按 public 处理（A2 白名单）

6.4 悄悄话 UI

- □ 目标条只在 `mode === "debate" && selected.length >= 2` 时出现
- □ 选中某人 → 输入框 placeholder 带该人名；发送按钮文案变；输入框边框变红
- □ 发送后消息带「私下 · 只有 X 能看见」标签，右侧气泡 + 红竖线
- □ 发送后目标条复位到「全体」（若采用复位方案）
- □ 把目标条选中的角色离座（点头像 `×`）→ 目标条自动回到「全体」
- □ 切到「对谈」模式 → 目标条消失、`whisperTo` 清空
- □ 手机端（≈375px）：目标条换行不挤压输入框

6.5 点名对峙

- □ 私聊甲说「乙不赞同你」→ 甲回复里点名乙 → 界面出现 `⚔` 提示条
- □ **下一轮乙第一个发言**
- □ 私聊甲但甲没点名 → 无 `⚔` 提示，下一轮顺序回到正常
- □ 甲点名自己 → 不产生 `summon` 事件
- □ 甲点名不在座的人（模型幻觉）→ 不产生 `summon` 事件，界面无异常
- □ 界面**从头到尾**看不到 `[[call:xxx]]` 字样（含流式过程中）
- □ `rounds === 1` + 私聊点名 → 本轮结束即收尾，`summon` 提示仍显示（不清不楚但无害）

6.6 流式与中断

- □ 「停止」（`page.tsx:1252`）在悄悄话回合中途按下 → 立即停，已说内容保留（`page.tsx:947-954`）
- □ 悄悄话回合中断后再发一条公开消息 → 上下文正常，无残留 `whisperTo`
- □ 连续快速发送（连点两次）→ `sendSeqRef`（`page.tsx:964`）阻止旧请求覆盖
- □ 「＋」新对话（`clearChat`，`page.tsx:831-842`）→ `whisperTo` 清空、`summonRef` 清空、`messages` 重置

6.7 存档与历史

- □ 含悄悄话的会话被存入 `mw_chat_history`（`page.tsx:812`）
- □ 历史列表标题**不含**悄悄话原文（B8）
- □ 恢复旧会话（点「继续」，`page.tsx:1212`）→ 悄悄话标签仍在，服务端仍按 `vis` 过滤
- □ 用改造前版本的 `localStorage` 数据 → 页面正常，所有消息按 public 渲染

- □ `/api/log` 的落盘文件里搜不到悄悄话原文（B9）

6.8 构建与类型

- □ `npx tsc --noEmit` 通过（`package.json` 无 typecheck 脚本，直接调 `npx tsc`）
- □ `npm run lint` 通过（`package.json:8`）
- □ `npm run build` 通过（Next 16 + React 19）
- □ `npm run dev` 起服务，用浏览器 DevTools 的 EventStream 面板核对事件序列

---

7. 风险与坑（按严重度排序）

🔴 致命

1. **`Msg.who` 是显示名，不是 id**（`page.tsx:922` 取 `ev.name`；服务端 `route.ts:286` 同时发了 `who:speaker.id` 但被丢弃）。

→ 任何指向具体角色的地方（`vis.to` / `whisperTo` / `summonRef` / `turnOrder`）**必须用 id**。必须在 `Msg` 上补 `whoId`（B5）并统一契约。搞错这一条，悄悄话会发给「名字恰好等于另一个 id」的鬼角色，或者永远过滤不掉。

1. **`history` 是模块级共享可变数组**（`route.ts:225` 定义、`route.ts:350` push）。

→ 改成 `rawHistory`（原始，含 vis）+ `buildHistoryFor(viewerId)`（每次过滤）后，**必须全量替换所有 `history` 引用**。漏一处就是悄悄话全场广播。

1. **`streamOnce` 逐字外发，无法回收**（`route.ts:280`）。

→ `[[call:id]]` 会在流式过程中被用户看到。必须「服务端剥离（`extractSummon`）+ 前端渲染期剥离（B10 `showContent`）」双保险。只做服务端剥离是不够的，因为剥发生在流结束后。

1. **单人对谈分支的触发条件是 `selected.length === 1`，不是 `mode`**（`route.ts:309`）。

→ 悄悄话回合**不能**通过 `characters: [target]` 表达，否则掉进单人分支，丢失 `roundRules` / `INFO_LAYER` / `SUMMON`，点名机制直接死掉。必须走 `whisperTo` 旁路 + `solo` 变量解耦（A2/A11）。

🟠 严重

1. **`memeBlock` 泄漏**（`route.ts:240-251`）：只算一次却拼进所有人的 system（`route.ts:311`、`330`）。v2 之后悄悄话会通过「最后一条非 assistant 消息」变成全员场外情报。**必须在 v2 里修**（A4）。

1. **点名顺序无跨请求状态**（`page.tsx:982` 每轮一次 HTTP）。

→ 服务端**不能**用模块级 Map 记「上轮点过谁」——多用户会串台（`searchBuckets`（`route.ts:58`）已是这种写法的反面教材）。顺序必须由前端 `summonRef` 保存并回传 `turnOrder`。

1. **`/api/log` 会把悄悄话原文上报**（`page.tsx:994-1003`）。用户输入的私密内容不该进站长日志。必须脱敏（B9）。

1. **`rawHistory.push` 无 `who`**（`route.ts:350`）：轮内发言无署名，跨轮有署名（来自 `route.ts:229` 的前缀）。

→ v2 若给轮内发言补 `who`，会**改变现有观感**并影响 `route.ts:319` 的反解正则。建议 v2 **先不补**，把「署名一致性」列为 v2.1 的独立改动 + 独立回归。

🟡 中等

1. **token 成本上涨**：每位发言人都要过滤全量历史，秘密越多 prompt 越大（12 条存档 × 多轮 × 多人）。要在 A4 之后实测单次请求体量与首字延迟。

1. **`roster` 拼进 system 会显著加长 prompt**（A9），12 人时尤其明显。建议 `selected.length > 6` 时关闭点名，或 roster 只列 id + 短名。

1. **名字反解正则脆弱**（`route.ts:319` `^([^\s：:]+)说[：:]\s*`）：只要 `Msg.who` 里出现空格或全角冒号，反解就失败 → 该条发言在 intl 下不会被换名。v2 不修，但**别在上面加更多依赖**。

1. **`I18N` 两侧必须同时补键**（`page.tsx:99-379`），否则 TS 报错或另一语言下显示 `undefined`。

1. **`charById` 的 TDZ**（`page.tsx:632` 声明）：任何在 `page.tsx:632` 之前使用它的派生值都会 ReferenceError（`const` + TDZ）。`whisperTarget` 要在渲染区算（B2 已标注）。

1. **`rounds` 语义变化**：悄悄话回合占掉 `rounds` 的第 1 轮。若用户设 `rounds=1`，一条私聊就只有一个人回话，用户可能以为「程序坏了」。**必须在目标条上给出明确文案**（`whisperHint`）。

1. **`summon` 事件与 `pushLive` 的时序**（`page.tsx:921`）：下一位发言人的 `who` 会固化上一位的 `live`。所以服务端必须**在下一轮 `ask` 之前**发 `summon`（A8 已保证），否则 `×` 提示会插错位置。

🟢 轻微

1. `key={i}`（`page.tsx:1225/1229`）用索引：插入 `auto` 消息会让后续 key 漂移。改用 `m.mid || i` 更稳，但要确认不破坏 `animationDelay` 的入场动画（`page.tsx:1229`）。

1. `ESCALATE_WORDS` / `maybeEscalate`（`page.tsx:53`、`844-859`）对悄悄话同样生效 —— 这是**有意为之**，写进验收，别当 bug 修。

1. `SavedChat` 没有 schema 版本号（`page.tsx:97`）。v2 靠「缺省即 public」容忍，暂不需要迁移代码；但若将来加 speaker→speaker 私聊，就必须加 `v` 字段。

1. 前端 `selected` 在 `send()` 的异步循环期间可能被用户拖动改变 → 顺序抖动（现存问题）。建议 `send()` 开头 `const seated = [...selected]` 快照。

1. 服务端 `temperature`（`route.ts:258`）按 `selected.length === 1` 分流 0.8 / 0.85。悄悄话回合在座 ⩾ 2 → 0.85，符合预期，不用改。但若产品希望「私房话更收敛」，可在此处按 `whisperTo` 单独给 0.75。

---

8. 明确不做（v2 范围外，避免膨胀）

- ❌ 发言者 → 发言者的私下对话（`vis.to` 指向另一个角色）。数据结构已支持（`Msg.vis` + `mid`），但 v2 只做「用户 → 角色」。
- ❌ 发言者的私下回复（`role:"assistant"` + `vis.kind === "private"`）。留接口不做。
- ❌ 前端隐藏私聊内容（用户是全知观众，必须看得见自己的悄悄话）。
- ❌ 数据库/服务端会话状态（一切靠前端全量重传 `messages`，无状态设计不能破）。
- ❌ 轮内发言补 `who` 署名（风险 8，留 v2.1）。
- ❌ `streamOnce` 首行暂存（A5，方案 B）—— 先用 B10 的渲染期剥离兜底。
- ❌ 点名并发/连点（一轮只解析第一个 `[[call:]]`）。

---

9. 实施顺序建议（依赖关系）

```
A1（类型/工具，无副作用）
 └ A2（入参 + speakers/solo）
     └ A3（buildHistoryFor 接线）          ← 此处先跑 6.1 + 6.2 回归，确认零变化
         ├ A4（memeBlock 堵洞）            ← 回归 6.3 第 1 项
         ├ A6（ask 扩展 whoId/summon）
         │   └ A7（单人分支改 speakers[0]）
         ├ A8（圆桌循环 speakers + 过滤 + 事件）
         │   ├ A9（prompt 两条纪律）
         │   └ A10（剥离 + summon 事件，已并入 A6/A8）
         └ A11（whisper 路由）
B1 → B5（契约先落：Msg 扩展 + whoId）
 └ B3（streamChat 新 body）→ B7（轮次循环）
     └ B4（SSE 分支）→ B2（state）
         └ B11 + B13 + B12（UI 入口/样式/文案）
             └ B6（userMsg 带 vis）→ B10（消息标记）
                 └ B8 + B9（存档与日志）
                     └ B14 + B15（口径与 changelog）
```

每个阶段的验收门槛：

1. `npx tsc --noEmit` + `npm run lint` 通过。
2. 6.1（对谈模式）全绿 —— 这是不可退让的红线。
3. 6.2（圆桌基线）全绿。
4. 进入下一阶段前，用 DevTools 手动核对**至少一次**完整的 SSE 事件序列。

本文档为只读侦察产出，未修改任何代码文件。

附录 A · 行号基线核验（必读）

本文档所有行号引用的是 git HEAD `517f2c0` **的版本**，已核验：

```
git show HEAD:app/page.tsx        | wc -l  →  1516   ✅ 与侦察时一致
git show HEAD:app/api/chat/route.ts | wc -l  →   374   ✅ 与侦察时一致
```

因此 §2 / §3 / §4 的行号是**改造前的权威基线**，可直接用于 code review 对照。

> ⚠️ **侦察期间发生了并发提交**：本文档成稿的同一分钟内（`page.tsx` mtime `00:36:07`，`route.ts` mtime `00:34:41`），**有另一个 agent 在工作区里落了 v2 的第一版实现并已提交**：
>
> ```
> 0bc0458 v2 信息层：可见性 + 悄悄话 + 点名拉人对峙（参赛版本 0.2.0） ← 新 HEAD
> 517f2c0 英文版彻底英文化：… ← 本文档的行号基线
> ```
>
> 当前工作区行数已变为：`page.tsx` 1550 行 / `route.ts` 479 行 / `revolution.css` 1019 行。
>
> **本文档的行号基线仍是 `517f2c0`**（未受污染，可用 `git show 517f2c0:app/page.tsx` 随时复核）。
>
> 因此本文档的价值变为二合一：既是「从 v1 基线出发的完整改动清单」，也是「对 `0bc0458` 第一版实现的差异审计」（见附录 B）。

附录 B · 工作区既有 v2 实现 vs 本文档清单（差异审计）

对 `git diff HEAD` 的实际内容做逐条比对。既有实现的核心设计：`Vis = "public" | string[]`（多收件人 id 数组）、服务端 `Utt` / `isPublic()` / `seesUtt()` / `visibleHistory(id)` / `whispersFor(id)`，请求体新增 `whisperOnly`，消息级新增 `whisperTo`。

B.1 逐条对照（26 个改动点）

编号	状态	说明
A1	✅ 等价	类型与工具齐备；但用 `Vis = "public" | string[]` 而非 {kind,to}，**不需要** SUMMON_RE / extractSummon`

编号	状态	说明		
A2	⚠️ 部分	只加了 `whisperOnly`；**无 `turnOrder`**（点名改在服务端同轮内解决，见 A10）		
A3	✅ 等价	`visibleHistory(speakerId)` 与原设计同义，且同样保留了 `X说：` 前缀逻辑		
A4	❌ **未做**	`memeBlock` **仍是全局**（`route.ts:302–308`，注入点 `381` / `401` / `425`）→ 见 B.2 第 1 条		
A5	❌ 未做	不需要（未采用标签协议）		
A6	❌ 未做	`ask()` 未扩展；`who` 事件**无 `whoId`**		
A7	✅ 等价	单人分支 `else if (selected.length === 1)` 原样保留，且 whisper 分支插在**它之前** —— 正确避开了 §7 风险 4 的 solo 陷阱		
A8	✅ 等价	`for (const speaker of selected)` → `while (queue.length) { queue.shift() }`；`visibleHistory(speaker.id)` 已接线		
A9	✅ 等价	`INFO_LAYER` + `WHISPER_NUDGE` 替代 `SUMMON` 协议（文案质量不错：明确要求「不要说『私下』『悄悄』这类字眼」）		
A10	⚠️ **换路线**	点名 = **回复文本里的人名子串匹配** `text.includes(s.zhName) \`	\	text.includes(s.name)`，queue.unshift()` **同轮内**提前。无标签、无 SSE 事件 → 见 B.2 第 2 条
A11	✅ 等价	`whisperOnly` 独立分支，语义与 `whisperTo` 一致		
B1	⚠️ 部分	`Msg` 只加了 `whisperTo?: string`；**无 `mid` / `whoId` / `vis` / `to`**		

编号	状态	说明
B2	✅ 等价	`useState<string>("")`（用空串而非 `null`）；**无** **`summonRef`**（不需要）
B3	✅ 等价	`streamChat(current, whisperOnly?)` + body 透传 `whisperTo` / `whisperOnly`
B4	❌ 未做	无新 SSE 事件（设计上不需要）
B5	❌ 未做	`Msg.who` 仍是显示名 → §7 风险 1 依然敞着
B6	✅ 等价	`userMsg = { role:"user", content:v, whisperTo: whisperTarget }`
B7	⚠️ **语义不同**	`rn = (mode==="debate" && !whisperTarget) ? rounds : 1` → 悄悄话回合**独占整次发送**，不再跑剩余圆桌 → 见 B.2 第 4 条
B8	❌ 未做	历史标题仍取 `firstUser.content`，悄悄话原文会进历史列表标题
B9	❌ **未做**	`/api/log`（`page.tsx:994-1003`）仍原样上传 → 悄悄话原文进站长日志
B10	✅ 等价	`rv-whisper` class + `rv-msg-whisper` 标签，中英双语
B11	⚠️ **选了方案 A**	「私」chip 挂在 `.rv-avatar` 上（本文档 §4.1 推荐方案 B 目标条）。同时用 `rv-whisper-bar` 作**状态提示条**（不是选择器）。功能可用，但 compact/mini 模式下的可用性未验证

编号	状态	说明
B12	❌ 未做	文案是**内联三元表达式**（`lang === "en" ? … : …`），未进 `I18N` 表 → 偏离项目既有约定
B13	✅ 等价	`revolution.css:975-1005` 已加 `.rv-avatar .wh`、`.rv-whisper-bar`、`.rv-msg.rv-whisper`、`.rv-msg-whisper`
B14	✅ 天然保留	`userCount` 未动，悄悄话照常计入气氛升级
B15	❌ 未做	changelog 未加条目

统计：✅ 12 项等价完成 · ⚠️ 5 项走了不同路线 · ❌ 9 项未做

B.2 既有实现里仍然敞着的洞（按严重度）

1. 🔴 `memeBlock` **泄漏未修**（本文档 A4）。

`route.ts:302-308` 仍以「最后一条非 assistant 消息」为触发源，只算一次，却拼进**全部三个分支**的 system（`381` / `401` / `425`）。 → 用户悄悄话说「XX 是什么梗」→ 检索结果作为「场外情报」注入**每位**发言人的 prompt。**这是唯一会真正泄漏悄悄话内容语义的洞**（`visibleHistory` 本身是对的）。 修法：触发源过滤成「仅 public 消息」，见 A4。**一处 3 行改动。**

1. 🔴 **人名子串点名会大量误触发**（替代 A10 的方案）。

`queue.findIndex((s) => text.includes(s.zhName) || (!!s.name && text.includes(s.name)))`：

- 哈耶克说「凯恩斯主义」→ 命中 `凯恩斯` → 凯恩斯被拉到队首；
- 中文模式下 `s.name`（英文名）也会参与匹配，跨语言误命中；
- 引用他人话语（「正如鲁迅所说」）同样触发。

→ 建议改为**显式协议**：要么用模型标签（本文档 A6/A9/A10，需双端剥离，见 §7 风险 3），要么把条件收紧为「`@` + 人名」或「人名 + 质问类动词」。当前实现在 demo 里会显得「随机拉人」。

1. 🟠 **悄悄话原文进** `/api/log`（本文档 B9 未做）。用户输入的私密内容流向站长后台。

1. 🟠 **悄悄话把** `rounds` **压成 1**（B7 走了不同语义）。

`rn = … ? 1 : rounds` → 用户设 4 轮，私聊后**只剩 1 轮**，圆桌交锋消失。叠加「同轮内点名提前」，用户实际只看到：「A 的私聊回复 + 被拉者的一句话」。与 v2 想要的「点名之后整桌重新交锋」有落差。建议改成 `r === 0` 单发言、后续轮照常（本文档 B7 原方案）。

1. 🟡 **死变量可能让 `npm run lint` 变红**：`route.ts:293` 的 `const history = visibleHistory(selected[0]?.id || "");` 已无引用（仅此一处 grep 命中），注释写着「兼容旧引用」但调用点都换成了 `visibleHistory(...)`。`eslint-config-next` 默认带 `no-unused-vars` 规则，建议直接删。

1. 🟡 **`Msg.who` 是显示名、无 `whoId`**（B5 未做）→ §7 风险 1 未消。当前靠服务端用 `selected` 里的 id 做 `vis` 目标，暂时没炸；但任何「按角色 id 回溯消息」的功能都会踩坑。

1. 🟡 **`whisperTo` 离座不清理**（B2 的 `useEffect` 未做）：目标人物被 `×` 离席后，UI 仍显示「正在私下对 …」，服务端 `whisperSpeakers` 过滤为空 → 静默降级为普通圆桌（不崩，但用户意图丢失且无提示）。

1. 🟡 **`setWhisperTo("")` 写在 `try` 块内**（B6/B7 附近）：`fetch` 抛错走 `catch` 时不会复位，界面会停在「私下」态，下一条公开消息会被误判成悄悄话。

1. **🟡 正则去掉了 `\s*`**：`debateContext` 的 `/^([^\s：:]+)说[：:]/`（原 `/…说[：:]\s*/`）配合英文 `": "` 前缀，会让 `slice(mm[0].length)` 残留一个前导空格。中文模式无影响。

1. 🟡 **I18N 未走表**（B12 未做）：新增 6+ 处 `lang === "en" ? … : …` 内联文案，双语维护成本上升，且与 `I18N[lang]` 的既有约定不一致。

B.3 给下一步行动的建议

- **立刻做**（3 行内、风险最低、收益最高）：修 `memeBlock` 触发源（B.2-1）、删 `route.ts:293` 死变量（B.2-5）。
- **本轮必做**：`/api/log` 脱敏（B.2-3）、`whisperTo` 离座清理 + `finally` 里复位（B.2-7/8）。
- **需要决策**：点名触发机制（B.2-2，误触发 vs 标签泄漏，二选一）；悄悄话是否独占整次发送（B.2-4）。
- **可以延后**：`whoId` / `mid` / `I18N` 收表 / changelog / compact-mini 模式下的「私」chip 可用性验证。

B.4 并发协作提醒（重要）

侦察期间，另一个 agent 已完成 v2 第一版并**提交为 `0bc0458`**（提交信息：「v2 信息层：可见性 + 悄悄话 + 点名拉人对峙（参赛版本 0.2.0）」）。

- **本文档的行号基线是 `517f2c0`（v1），不是当前 HEAD。** 用行号定位代码时：
- 看改造前的原貌 → `git show 517f2c0:app/page.tsx | sed -n '225,231p'`
- 看当前实现 → 行号已整体偏移（`page.tsx` +34 行、`route.ts` +105 行），**不要直接套用本文档行号**。
- 附录 B.1 的对照表**已包含 `0bc0458` 的全部内容**，是当前唯一的权威差异视图。

- 下一轮改动请从附录 **B.3** 的清单出发，不要再从 §2/§3 的「改成什么」重做一遍 —— 那会覆盖已有实现。
- 若需要回退到基线重新实现：`git reset --hard 517f2c0`（⚠️ 会丢弃 `0bc0458`，先确认 `competition/` 下的文档已单独备份）。